

ISG 5311 - Genomic Data Analysis in Practice

Noah Reid

2023-09-14

Table of contents

Preface	6
Materials necessary for the course	6
 I Intro to Linux and Bash	 8
 II Learning Objectives	 9
 1 Getting Connected	 11
1.1 Objective	11
1.2 Getting ready to connect with MacOS	11
1.3 Getting ready to connect with Windows	11
1.4 Connecting	13
1.4.1 Breaking it down	13
1.5 Linux Commands in This Section	15
 2 Getting Started with Linux	 16
2.1 The Linux operating system	16
2.2 The shell and the operating system	16
2.3 Navigating the file system	17
2.3.1 Paths	18
2.3.2 Changing the working directory	20
2.3.3 Listing directory contents	20
2.3.4 The <code>find</code> command	23
2.4 Manipulating files and directories	24
2.4.1 Creating directories	24
2.4.2 Creating files	25
2.4.3 Moving and copying	26
2.4.4 Removing things	28
2.5 Ownership and Permissions	30
2.5.1 Changing permissions	31
2.5.2 Changing groups	32
2.6 Getting Help	32
2.6.1 Built-in help	32

2.6.2	The Internet!	33
2.6.3	AI/large language models	33
2.7	Asking questions about the system	34
2.8	Basic Linux Commands	35
2.9	Exercises	35
2.9.1	Practical exercise:	35
2.9.2	Quiz: See Blackboard Ultra for this section's quiz.	37
3	Working with Files	38
3.1	Moving files between computers	38
3.1.1	<code>wget</code> and <code>curl</code>	38
3.1.2	<code>scp</code>	39
3.1.3	<code>rsync</code>	39
3.1.4	FTP clients and Globus	40
3.2	File compression	40
3.3	Basic file inspection	41
3.3.1	<code>less</code>	42
3.3.2	<code>head</code> , <code>tail</code> and <code>cat</code>	42
3.3.3	Counting with <code>wc</code>	43
3.3.4	<code>cut</code>	43
3.4	Regular expressions and <code>grep</code>	43
3.5	STDIN, STDOUT, STDERR, redirection and the pipe	45
3.5.1	The pipe	47
3.6	<code>sed</code>	48
3.6.1	Replacements	48
3.7	Line printing	49
3.8	<code>awk</code>	49
3.9	Linux Commands in This Section	51
3.10	Exercises	51
3.10.1	Quiz:	51
	References	52
III	HPC and SLURM	53
IV	Learning Objectives	54
4	Scripts and putting it all together	56
4.1	One-Liners	56
4.2	Environment variables	57
4.2.1	Invoking variables	57

4.2.2	Setting variables	57
4.2.3	An aside about quotes in BASH	58
4.2.4	The PATH variable	59
4.2.5	Generalizing code	59
4.3	Loops	60
4.3.1	for loops.	60
4.3.2	while loops.	62
4.4	Editing Files	63
4.4.1	Command-line text editors	63
4.4.2	Code editors	64
4.4.3	Integrated development environments.	64
4.4.4	Conveniently accessing files.	64
4.5	Scripts	66
4.5.1	The shebang	66
4.5.2	Comment lines	66
4.5.3	Executing scripts	67
4.5.4	Errors and exit codes	69
4.6	Exercises	70
5	SLURM: the job scheduler	71
5.1	High performance computing review	71
5.2	SLURM	72
5.2.1	The general approach	72
5.2.2	What resources are needed?	72
5.2.3	What resources exist/are available?	73
5.2.4	Request resources (and submit work)	76
5.2.5	Monitor job progress	79
5.2.6	Evaluate the work	81
5.2.7	What resources <i>should</i> I request?	82
5.3	Exercises	83
6	Data Storage and Software on HPC	84
6.1	Data storage and access	84
6.2	Software	84
V	Sequencing Data	85
7	Untitled	86
8	seq data	87

VI RNA-seq	88
9 Untitled	89

Preface

This course is focused on imparting practical knowledge of how to conduct bioinformatic analyses, with a focus on teaching users to interact with computers using a command-line interface. The initial parts of the course will introduce you to the Linux operating system, the BASH shell, and how to access and schedule work on a high performance computing cluster. We will move from there to understanding sequencing data and exploring ways to manipulate and summarize it. Finally, we will explore a very common workflow, differential expression with bulk RNA-seq, as a model for how to organize and conduct an analysis project. In the last stage we will also introduce the R statistical computing language, and use differential expression analysis as a vehicle for learning about data visualization.

Materials necessary for the course

You will need a laptop or desktop computer running a Windows, MacOS or Linux operating system. Most machines produced in the last few years will be sufficient because heavy computation will be done on a powerful remote computer cluster.

You'll be working at your a computer a lot during this course, often simultaneously using multiple applications, so it will be helpful to have the right setup. Screen real estate is an important factor. If you're working at a desktop, the bigger the display(s) the better. If you are using a laptop, we recommend a desk space with a second display, mouse and keyboard, if possible. Something like this:



Figure 1: A useful, though maybe not aesthetically pleasing, work station

Part I

Intro to Linux and Bash

Part II

Learning Objectives

Learning Objectives:

Connect to a remote computer cluster
Navigate the Linux file system using the BASH shell
Manipulate files and directories
Modify file ownership and permissions
Use multiple strategies to get help and problem solve
Ask questions about the status of the system
Move files between computers
Work with file compression
Inspect files
Use regular expressions with **sed** and **grep**
Pipe inputs and outputs

Resources:

- <https://www.gnu.org/savannah-checkouts/gnu/bash/manual/bash.html>
- <https://tldp.org/LDP/abs/html/index.html>
- <https://journals.plos.org/ploscompbiol/article?id=10.1371/journal.pcbi.1008645>
- <https://journals.plos.org/ploscompbiol/article?id=10.1371/journal.pcbi.1009207>
- <https://github.com/onceupon/Bash-Oneliner>

1 Getting Connected

Learning Objectives:

Connect to the Xanadu cluster

Identify when a terminal window is connected to a remote computer

1.1 Objective

Our starting exercises, and indeed most of the work for the course, will take place on a remote computer cluster running a Linux operating system (the cluster is named “Xanadu”). So the first thing you’re going to do is connect your local machine to that server. We’ll get into many of the details to help you understand what’s going on later, but to start we’re going to have you follow some directions without a lot of explanation.

You can connect to the server using a computer running Windows, MacOS, or any variety of Linux. We’ll cover how to do it with a machine running Windows or MacOS here, and assume that if you are running Linux as your primary operating system, you are savvy enough that you can piece this together on your own (or ask an instructor for help).

1.2 Getting ready to connect with MacOS

Start by opening the app “Terminal” on your computer. It should already be installed. MacOS is a unix-like operating system (as is Linux) and as such, you can interact with it through the command-line in a terminal without installing any special software (again, more on this later). When you open a terminal, you’ll see a mostly empty window with a **prompt**. The prompt will be a single symbol, probably a `>` or a `$`. Here is an example of a terminal with a customized prompt (giving the username and current working directory):

1.3 Getting ready to connect with Windows

Connecting to a Linux-based cluster is a little more complicated on a machine running Windows than one running MacOS. There are a few different ways to do it, but our recommended

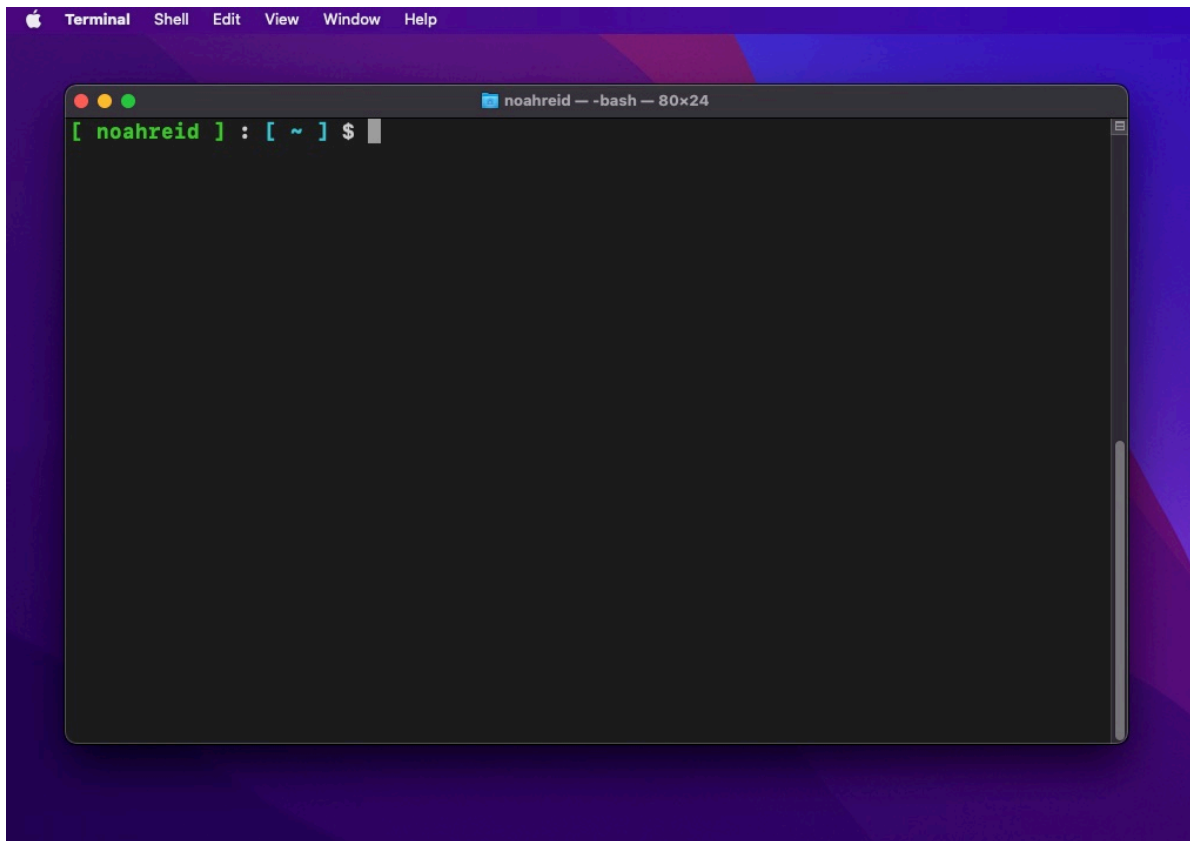


Figure 1.1: A Terminal Window

approach is to use Windows Subsystem for Linux (WSL). WSL essentially allows a Windows machine to run Linux programs, and interact with the Linux operating system through a command-line terminal. Connecting to the cluster this way will mean that most future instruction will be identical, or nearly so, for Mac and Windows users.

WSL first needs to be installed. Instructions for that are [here](#). There are different flavors of Linux: Debian, Ubuntu, CentOS, which Xanadu runs on, and many others. the WSL default, Ubuntu, will be fine.

After you've installed WSL, you should open a Ubuntu terminal window. When you open a terminal, you'll see a mostly empty window with a **prompt**. The prompt will be a single symbol, probably a `>` or a `$`. Above is an example of a terminal with a customized prompt (giving the username and current working directory).

1.4 Connecting

You should now have a terminal window open. We're going to use SSH (*Secure SHell* protocol) to connect to Xanadu. You should already have set up your Xanadu account with a username and password (if not, see [here](#)). The username is distinct from your netID, and is usually first initial, last name (i.e. the username for John Smith will often be *jsmith*).

To login type the following command at the prompt:

```
ssh username@xanadu-submit-ext.cam.uchc.edu
```

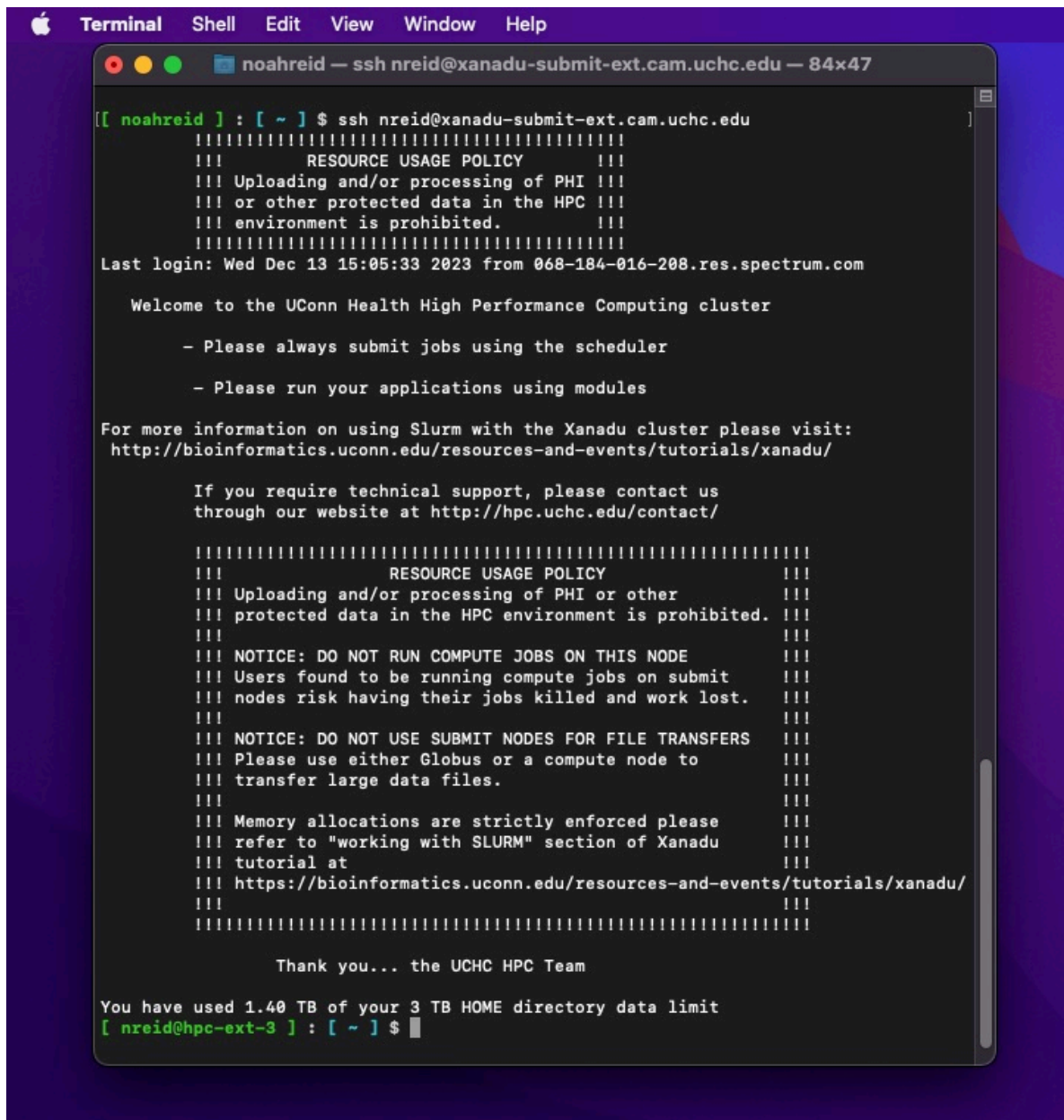
You will be prompted for your password. When you type your password, the cursor will not move, and you won't see any characters on the screen. This is normal. Just hit enter when you've finished typing your password.

If you were successful and made no mistakes your terminal window is now connected to a remote computer (i.e. Xanadu) and you should see a screen like this:

Again, this particular terminal prompt has been customized (we'll show you how to do that later) so your prompt won't look the same, but you have now logged in to the Xanadu cluster.

1.4.1 Breaking it down

Because this is our first command issued in the course, I want to break down what we did. When you typed `ssh username@xanadu-submit-ext.cam.uchc.edu`, you used the fundamental command-line syntax `<command> <arguments>`. You told **your computer** to execute the program `ssh`, which allows you to securely gain command-line access to another computer, and provided the details ("arguments") `username@xanadu-submit-ext.cam.uchc.edu`, which are your username, and the server address of the computer you want to connect to.

A screenshot of a macOS Terminal window. The title bar shows 'Terminal' and standard window controls. The window title is 'noahreid — ssh nreid@xanadu-submit-ext.cam.uchc.edu — 84x47'. The terminal content shows a user logging in via SSH. The prompt is '[noahreid] : [~] \$'. The user enters 'ssh nreid@xanadu-submit-ext.cam.uchc.edu'. The terminal displays a series of messages: a separator line of exclamation marks, a 'RESOURCE USAGE POLICY' warning about PHI, a 'Last login' timestamp, a 'Welcome to the UConn Health High Performance Computing cluster' message, two bullet points about using the scheduler and modules, a link to Slurm documentation, technical support contact info, another 'RESOURCE USAGE POLICY' warning, a 'NOTICE: DO NOT RUN COMPUTE JOBS ON THIS NODE' warning, another 'NOTICE: DO NOT USE SUBMIT NODES FOR FILE TRANSFERS' warning, memory allocation rules, and a final URL. It ends with 'Thank you... the UCHC HPC Team' and a data limit notice: 'You have used 1.40 TB of your 3 TB HOME directory data limit'. The final prompt is '[nreid@hpc-ext-3] : [~] \$' with a cursor.

```
[ noahreid ] : [ ~ ] $ ssh nreid@xanadu-submit-ext.cam.uchc.edu
!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!
!!!          RESOURCE USAGE POLICY          !!!
!!! Uploading and/or processing of PHI !!!
!!! or other protected data in the HPC !!!
!!! environment is prohibited.          !!!
!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!
Last login: Wed Dec 13 15:05:33 2023 from 068-184-016-208.res.spectrum.com

Welcome to the UConn Health High Performance Computing cluster

- Please always submit jobs using the scheduler

- Please run your applications using modules

For more information on using Slurm with the Xanadu cluster please visit:
http://bioinformatics.uconn.edu/resources-and-events/tutorials/xanadu/

If you require technical support, please contact us
through our website at http://hpc.uchc.edu/contact/

!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!
!!!          RESOURCE USAGE POLICY          !!!
!!! Uploading and/or processing of PHI or other !!!
!!! protected data in the HPC environment is prohibited. !!!
!!!                                     !!!
!!! NOTICE: DO NOT RUN COMPUTE JOBS ON THIS NODE !!!
!!! Users found to be running compute jobs on submit !!!
!!! nodes risk having their jobs killed and work lost. !!!
!!!                                     !!!
!!! NOTICE: DO NOT USE SUBMIT NODES FOR FILE TRANSFERS !!!
!!! Please use either Globus or a compute node to !!!
!!! transfer large data files.          !!!
!!!                                     !!!
!!! Memory allocations are strictly enforced please !!!
!!! refer to "working with SLURM" section of Xanadu !!!
!!! tutorial at          !!!
!!! https://bioinformatics.uconn.edu/resources-and-events/tutorials/xanadu/
!!!                                     !!!
!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!

Thank you... the UCHC HPC Team

You have used 1.40 TB of your 3 TB HOME directory data limit
[ nreid@hpc-ext-3 ] : [ ~ ] $
```

Figure 1.2: The usual Xanadu login message

I want to take a moment to emphasize something that might not be entirely clear yet, but which is very important. When you first opened your terminal window, any commands you wrote there were commands directed to and executed by **your computer**. After you ran `ssh` on your computer to connect to Xanadu, your terminal window effectively became a terminal window on the **Xanadu cluster**. Any subsequent commands you enter at the prompt will be received and executed by **Xanadu**. It is as if you were actually sitting at a terminal physically connected to one of Xanadu's nodes. Nothing you do at this terminal will impact your computer in any way, and programs running on Xanadu cannot directly access your local system or any of its files. Conversely, nothing you do outside this terminal window will impact Xanadu.

Unless you are told directly, all the work for the Linux and HPC sections of this course should be conducted on Xanadu, after using `ssh` to connect.

Now, without a customized prompt, it will not always be obvious *what computer* a given terminal is connected to. If you're working in a terminal window and you're not sure about this, you can always enter the command `hostname`. If the output is something like `hpc-ext-#`, `hpc-int-#`, or `xanadu-##`, then you're connected to Xanadu. If it's anything else you're not! Try that now. This is especially useful because in some cases you may want to have multiple terminal windows open at a time.

Ok, now that we're connected, this exercise is nearly over. You can disconnect by typing `exit`. You should see a message like this:

```
logout
Connection to xanadu-submit-ext.cam.uchc.edu closed.
```

Type `hostname` to verify that you are working locally. Now any commands you type in the terminal will be executed by your computer. Alternatively, you can simply close the terminal window, and that will kill your connection to Xanadu.

By now you've probably noticed the terms **remote** and **local** being used regularly. When we refer to doing something *locally*, we mean to do it on your personal computer, without connecting it to Xanadu, whereas *remote* means the opposite.

1.5 Linux Commands in This Section

Command	Description
cmnd	desc

2 Getting Started with Linux

Learning Objectives:

Navigate the Linux file system using the BASH shell
Manipulate files and directories
Modify file ownership and permissions
Use multiple strategies to get help and problem solve
Ask questions about the status of the system

2.1 The Linux operating system

As we have mentioned, this course will make heavy use of a high performance computing cluster running a **Linux operating system**. But what is Linux? [Linux](#) is an open-source computer operating system descended from an older operating system called [Unix](#). There are lots of different flavors of Linux, referred to as **distributions**.

The vast majority of cutting-edge research software is built in Linux environments, and can be relatively easily compiled on most distributions. MacOS is built on a different descendant of Unix, [BSD](#), which differs from Linux. Because of their shared ancestral design features, however, most software developed for use on Linux systems can be compiled and run on MacOS. To run Linux software on a Windows system, a good option is to install *Windows Subsystem for Linux*, which we asked you to do in the previous chapter if you are using Windows. Xanadu runs on a Linux distribution called [CentOS](#). This is occasionally important to know when compiling software.

Linux distributions can have *graphical user interfaces*, but for scientific computing purposes, interacting with Linux through a **command-line interface** (sometimes abbreviated **CLI**) is nearly universal.

2.2 The shell and the operating system

While Linux is the **operating system** that runs the computer, command-line interaction with that operating system happens through another layer of software called a **shell**. A shell is essentially an application, like any other, that takes input from the user, passes it to the

operating system to be processed, and then returns any output, warnings or errors. There are several commonly used command-line shells, including Z shell (zsh), TC shell (tcsh) and Bourne-Again shell (bash). These shells share many features in common, but in scientific computing, **bash** is most commonly used and is what we'll use here. Newer MacOS computers default to zsh (this can be changed), but zsh is an extension of bash that is mostly compatible with it, so bash scripts should work in zsh.

Try this now: To find out what shell your local computer is running, open a terminal window and type `echo $SHELL` at the prompt. `echo` is a command that simply prints whatever input it receives and `$SHELL` is an environment variable (more about those later). The variable contains a character string, probably `/bin/bash` or `/bin/zsh`, depending on your operating system and if you have changed the defaults. If you log in to Xanadu (remember: `ssh username@xanadu-submit-ext.cam.uchc.edu`) and type the same command, it will say `/bin/bash`.

Your interactions with the command-line shell will nearly always take the form of `<command> <argument1> ... <argumentN>`. You've seen this twice now when you ran `ssh` and `echo`. Using the command line, you will issue commands to navigate the file system, to manipulate files, and to execute programs.

2.3 Navigating the file system

Now that we have learned a little about the Linux operating system and how to connect to a remote computer, a natural place to begin *interacting* with Linux is to learn how to navigate the **file system**. The file system organizes and tracks data stored on a computer. It keeps track of where files are on a storage device, how big they are, when they were created, which users own them, and many other things.

These days, many people's primary experience with computers is through smartphones. Smartphones running the IOS and Android operating systems obscure and restrict access to the file system. On a smartphone, your entire interaction with the OS is through apps, of which the phone's graphical user interface (GUI) is one. For example, if you want to explore, edit, copy, or delete photos you've taken, you might use Google Photos, or some other specialized app. If you want to listen to music files you've downloaded, you might use Spotify. At no point do you, personally, manipulate any of these files.

GUIs and CLIs on personal computers, on the other hand, often give you more flexibility in access to the filesystem. You might use a photo app to explore or edit photos on your laptop, but if you wished to copy, move, or delete a photo, you could do that yourself by navigating through the filesystem to the place the photo is stored and then directly acting on the file through the operating system, rather than by asking a specialized app to do it for you.

In a GUI, the file system itself is usually represented as a nested series of directories, graphically depicted as windows. You open the window for a directory, and inside you will see files and

subdirectories, possibly in the form of a list, or maybe as a series of icons laid out in a grid. There are also likely to be directories which you can open up in new windows by pointing the mouse at them and double-clicking. You may have multiple windows open at once, but only one of these will be the window in which you are actively working. If you use a keyboard shortcut or menu option to create a new file or directory, that action will occur in the active window.

In a Linux CLI, the file system is also represented by a hierarchical directory structure, where each directory contains files and often sub-directories. There is, however, typically no graphical visualization of that structure available. Though you cannot see that structure in the form of windows, you still have something analogous to the “active window” in a GUI, a *working directory*. If you issue commands that create or modify files or directories, their effects will by default occur in your current working directory.

2.3.1 Paths

When doing any kind of work on a Linux CLI, you’ll need to be aware of what the current working directory is. There are two ways to access it. First, there is a simple command, `pwd` (for **p**rint **w**orking **d**irectory). Second, there is a built-in environment variable, `$PWD`.

Try this now: Log in to Xanadu and try entering `pwd` and `echo $PWD` now. Both will return something like the following:

```
/home/FCAM/username
```

This is called a *path*. It gives the full sequence of enveloping directories for the current working directory, with each directory name separated by `/`, and ending in the current working directory with name `username`. The initial `/` refers to the *root directory*, which has no name and no parent directories. `home` is one of the directories found in root, `FCAM` is found inside `home`, and `username` is found inside `FCAM`. If you change your current working directory, the results of `pwd` and `echo $PWD` will change as well.

Assuming you executed the above commands after logging in, and before doing anything else, *the particular path* you saw is pretty important. It is your **home directory**. Xanadu, like most high performance computing clusters, has many users, and each has a home directory where only they have permission to make or modify files and directories. Because of the special status of this directory, it is also assigned a symbolic shortcut: `~`.

Try this now: Type `echo ~`. No matter what your current working directory is, `~` will always be set to your home directory.

Paths that begin with `/`, like `/home/FCAM/username` are referred to as **absolute paths** (or full paths), because they give the complete sequence of directories from the root to the target file or directory. There is no ambiguity about what an absolute path refers to on the file system.

We can also specify paths **relative** to the current working directory. Let's say your current working directory contains a project directory, `killifishGenomes` with two subdirectories `rawdata` and `scripts` that looked like this:

```
killifishGenomes/  
  rawdata  
  scripts
```

The absolute path to the `rawdata` directory looks like this:

```
/home/FCAM/username/killifishGenomes/rawdata
```

If your current working directory is `~`, however, you can equally well specify the directory with paths that are *relative* to your working directory like this:

```
killifishGenomes/rawdata
```

or this:

```
./killifishGenomes/rawdata
```

where the `.` stand in for the current working directory.

What if your current working directory is `scripts`, and you want to refer to `rawdata`? You can use `..`, to indicate the relative path goes *up* one level in the hierarchy before descending into another directory:

```
../rawdata
```

Imagine our directory structure was a little more complex, like this:

```
killifishGenomes/  
  rawdata  
    ont  
    pacbio  
  scripts  
    assembly  
    QC
```

Say we are working in the directory `scripts/QC` and we want to point to a file in `pacbio`. The **absolute path** is `/home/FCAM/username/killifishGenomes/rawdata/pacbio` and the path **relative** to our current working directory is `../../rawdata/pacbio`, indicating the path ascends two levels in the directory hierarchy, then descends two levels to `rawdata/pacbio`

Note

Relative paths can be useful in two ways. First, they are often much shorter than full paths, making scripts easier to read. Second, if you are working in a project directory that you need to move, perhaps from `/home/FCAM/username` to `/labs/principalinvestigator`, because you are running out of space in your home directory, scripts containing absolute paths (like `/home/FCAM/username/killifishGenomes/rawdata/pacbio`) pointing inside the project directory will break (i.e. point to locations in the file system that no longer exist), while those with relative paths (like `../../rawdata/pacbio`) will still function.

2.3.2 Changing the working directory

All of the above discussion of paths and working directories raises the question: How do I change my working directory? The answer is to use the command `cd`, which takes a single argument: the directory you wish to move into.

Try this now: Making sure you are logged in to Xanadu, try changing to the root directory, then type `pwd`:

```
cd /  
pwd
```

Remember, `/` refers to the root. Now let's navigate to your home directory one step at a time, typing `pwd` after each step:

```
cd FCAM  
pwd  
cd username  
pwd
```

If you forget where your home directory is, remember the shortcut `~`. `cd ~` will always take you home.

You can use relative and absolute paths to change directories, as we discussed above.

2.3.3 Listing directory contents

A downside of command-line navigation of the filesystem is that you don't immediately see directory contents as you do in a GUI with windows. For that, we use the command `ls`. Since we are just getting started, your home directory on Xanadu is probably empty, so if you go there and type `ls`, you should get no results.

Let's go somewhere more interesting.

Try this now: List the contents of the directory `/isg/shared/apps/`. You can do this one of two ways. Without changing the working directory, and supplying a target directory as an argument to `ls`:

```
ls /isg/shared/apps
```

Or you can move to that directory and type `ls` with no arguments:

```
cd /isg/shared/apps
ls
```

The directory contents should be a very large list. This directory contains a subdirectory for each globally installed (i.e. available to all users) software package on Xanadu. Each subdirectory contains yet more subdirectories for each version of the package installed. Let's look more closely at `iqtree`, a package used for phylogenetic inference.

```
cd /isg/shared/apps/iqtree/
ls
```

At the time of writing, there were four subdirectories, corresponding to 4 package versions:

```
1.5.5  1.6.10  1.6.6  2.1.3  2.2.2
```

Let's visit 2.2.2 and have a closer look:

```
cd 2.2.2
ls
```

You should see:

```
bin  example.cf  example.nex  example.phy  models.nex
```

Ok, great. We see some things in this directory, but can't we learn a little more about them? Yes, by supplying some arguments to `ls`.

The first let's try the argument `ls -l`. This will give you the *long form* directory listing, which looks like this:

```
total 2364
drwxr-xr-x 2 root root    512 Dec 14  2022 bin
-rw-r--r-- 1 root root 2237314 Nov 15  2022 example.cf
-rw-r--r-- 1 root root   174 Nov 15  2022 example.nex
```

```
-rw-r--r-- 1 root root 34178 Nov 15 2022 example.phy
-rw-r--r-- 1 root root 121669 Nov 15 2022 models.nex
```

We won't cover all this in detail just yet, but some key information is

- Column 1: The file permission string. The first letter (d or -) tells you whether the item is a directory or a regular file.
- Column 5: The file size (in bytes).
- Column 6,7,8: The file modification date.

For easier-to-read file sizes, supply a second argument `ls -l -h` or the more abbreviated combination of arguments `ls -lh`.

```
total 2.4M
drwxr-xr-x 2 root root 512 Dec 14 2022 bin
-rw-r--r-- 1 root root 2.2M Nov 15 2022 example.cf
-rw-r--r-- 1 root root 174 Nov 15 2022 example.nex
-rw-r--r-- 1 root root 34K Nov 15 2022 example.phy
-rw-r--r-- 1 root root 119K Nov 15 2022 models.nex
```

Now the file sizes have a suffix indicating the unit (K = kilobytes, M = megabytes, G = gigabytes)

There are a few more arguments that may come in handy:

- `-t` sorts the listing by modification date
- `-R` lists *recursively*, meaning it will also list the contents of subdirectories.

I want to introduce another concept here, the *glob* : `*`. This symbol serves as a *wildcard* when referring to files and directories. If you want to list only the files in the directory beginning with “example” you can type `ls -lh example*`

```
-rw-r--r-- 1 root root 2.2M Nov 15 2022 example.cf
-rw-r--r-- 1 root root 174 Nov 15 2022 example.nex
-rw-r--r-- 1 root root 34K Nov 15 2022 example.phy
```

You can use multiple globs, if necessary. `ls -lh *l*` will output every element containing the letter “l”.

```
-rw-r--r-- 1 root root 2.2M Nov 15 2022 example.cf
-rw-r--r-- 1 root root 174 Nov 15 2022 example.nex
-rw-r--r-- 1 root root 34K Nov 15 2022 example.phy
-rw-r--r-- 1 root root 119K Nov 15 2022 models.nex
```

Note

There is one more way of looking at directory contents, but it's not available on all systems: `tree`. It will print a nicely formatted visualization of directory structures. Try it. You should see:

```
.
├── bin
│   ├── iqtrees2
│   ├── example.cf
│   ├── example.nex
│   ├── example.phy
│   └── models.nex
└── 1 directory, 5 files
```

For complex directories you can supply the argument `-L` and a number to specify the number of directories deep the listing should go.

2.3.4 The `find` command

It's often the case that you will want to find files in a large and complicated directory. Linux does not have an indexed search feature like Spotlight in MacOS, so if you're looking for files by name, or content within files you'll need other strategies. Here we'll discuss how to find files by name using `find`.

`find` is an extremely flexible utility included with Linux. It allows searching for files within a specified directory on combinations of attributes such as name, modification date, size, owner and more. Here we'll just cover the simple case of searching by name. The most basic usage of `find` is `find <path to search> <expression>`. The expression we're going to use is `-name "matchpattern"`. The match pattern can use one or more globs as a wildcard.

Try this now: We can search for all files with the suffix `fastq.gz` in the directory `/core/cbc/tutorials/workshopdirs/RNA-seq-with-reference-genome-and-annotation` like this:

```
find /core/cbc/tutorials/workshopdirs/RNA-seq-with-reference-genome-and-annotation -name "
```

`Find` needs to traverse all the subdirectories of a target, so in a very large and complicated project, it can be somewhat slow.

2.4 Manipulating files and directories

We have seen some basic ways to navigate the file system. Now we're going to start looking at how to examine and manipulate files and directories.

2.4.1 Creating directories

We're going to start with the basics of creating things and moving them around, and worry about the contents of files later.

Try this now: To start, let's create a series of directories and populate them with (empty) files as a demonstration. Make sure you are logged in to Xanadu and in your home directory (remember `hostname` and `pwd` to check if you're not sure). Enter the following commands at the prompt:

```
mkdir killifishGenomes
mkdir killifishGenomes/scripts
mkdir killifishGenomes/rawdata
mkdir killifishGenomes/results
```

The command `mkdir` creates a directory with the name you provide. This will create a directory structure like this:

```
killifishGenomes/
  rawdata
  results
  scripts
```

You can create multiple directories in a single command by providing multiple arguments. Perhaps the most succinct way of creating all these directories at once is with a **shell expansion**. This is a feature of the bash shell that allows lists or ranges of values to be expanded. Let's first `echo` some shell expansions just to see what it looks like. Enter the following on the command line:

```
echo {a..z}
echo {1..10}
echo {scripts,rawdata,results}
echo killifishGenomes/{scripts,rawdata,results}
```

So, to create all these directories with `mkdir` you could simply have written:


```
mkdir -p killifishGenomes/{scripts,rawdata,results}
```

The `-p` flag means to create any parent directories in the path as needed.

i Note

Let's briefly introduce you to some errors.

Try the following, typed **exactly**:

```
mkdir mish/mash
```

You got an error, right? Which argument above could have solved this for you?

Now try this, typed **exactly**

```
mkdir -p mish /mash
```

You should have gotten another error. The space means `mish` and `/mash` are two separate arguments. `mkdir` created `mish` with no problem (try `ls` and you'll see), but `/mash` couldn't be created because regular users don't have permission to write in the root directory, which, if you remember, is signified by a leading `/` on any path.

This being the beginning of your journey into bash and Linux, you will encounter many, many errors. We'll talk more about them later.

2.4.2 Creating files

There are several ways that files can be created. A very simple way is by redirecting output into a new file.

Try this now: Let's use the directory structure you created above to do this. Assuming you created the directory `killifishGenomes` in your home, and your home is your current working directory:

```
echo "This is a test directory" >killifishGenomes/README.md
```

This will write the text you **echoed** to the file `README.md`. The `>` symbol redirects output to a file. If the file already exists, it will be overwritten. We'll talk in more detail about that later.

You can *append* new lines to that file with `>>`:

```
echo "There aren't actually any genomes in here" >>killifishGenomes/README.md
```

We're going to cover more on inspecting files later, but for now, check that your commands were successful by using the command `cat`, which will write the contents of the file to the terminal:

```
cat killifishGenomes/README.md
```

We can also create empty files using the command `touch`. I don't use `touch` often, except in demonstrations like this. You can create many files using shell expansions:

```
touch killifishGenomes/scripts/{QC,assembly}.sh
touch killifishGenomes/rawdata/sample{1..5}.fastq.gz
touch killifishGenomes/results/sample{1..5}.fasta
```

Note that above we used a range of numbers in our shell expansion, `{1..5}`, to create 10 empty files. Shell expansions can also be used with letters, e.g. `{a..z}`. Check to see that the files are where you expect them to be using `ls`.

If you've been following along so far, you can also try typing `tree killifishGenomes`. You should see this structure:

```
killifishGenomes/
  rawdata
    sample1.fastq.gz
    sample2.fastq.gz
    sample3.fastq.gz
    sample4.fastq.gz
    sample5.fastq.gz
  README.md
  results
    sample1.fasta
    sample2.fasta
    sample3.fasta
    sample4.fasta
    sample5.fasta
  scripts
    assembly.sh
    QC.sh
```

2.4.3 Moving and copying

You will often need to move or copy files and directories within a system. There are two key commands we use for these tasks: `mv` for moving, and `cp` for copying.

Let's say you've written some scripts, but you can see that as your project grows, your scripts directory is going to become crowded and feel disorganized. One solution is to move scripts into subdirectories. `mv` takes two arguments: a path to a source file or directory, and a path to a destination directory.

Try this now: From your home directory type:

```
mkdir -p killifishGenomes/scripts/{assembly,QC}
mv killifishGenomes/scripts/assembly.sh killifishGenomes/scripts/assembly
mv killifishGenomes/scripts/QC.sh killifishGenomes/scripts/QC
```

You can optionally rename things as you move them. If you wanted to rename the QC directory, you can simply append a new name to the path. Try it now:

```
mv killifishGenomes/scripts/QC killifishGenomes/scripts/quality_control
```

If you've followed these steps, your directory structure will now look like this:

```
killifishGenomes/
  rawdata
    sample1.fastq.gz
    sample2.fastq.gz
    sample3.fastq.gz
    sample4.fastq.gz
    sample5.fastq.gz
  README.md
  results
    sample1.fasta
    sample2.fasta
    sample3.fasta
    sample4.fasta
    sample5.fasta
  scripts
    assembly
      assembly.sh
    quality_control
      QC.sh

5 directories, 13 files
```

Copying works similarly to moving, but the original copy of the file or directory remains in place. You provide a source file or directory and a destination. You can rename as you copy as well. Let's move to the scripts directory for this step and try copying some things:

```
cd killifishGenomes/scripts
cp assembly/assembly.sh assembly/assemblyV2.sh
cp assembly assembly_flye
```

You should have seen an error when trying to copy the directory `assembly`. To copy an entire directory you need to use the flag `-r`:

```
cp -r assembly assembly_flye
```

Your scripts directory should now look like this:

```
assembly
  assembly.sh
  assemblyV2.sh
assembly_flye
  assembly.sh
  assemblyV2.sh
quality_control
  QC.sh
```

Warning

If you copy, move or rename a file, and a file with that name already exists at the destination, **the pre-existing file at the destination will be overwritten.**

2.4.4 Removing things

Here is where things start to get a bit dangerous. To permanently delete files in a GUI like MacOS or Windows, you usually must take several very explicit, specific actions, and the OS often asks you if you're certain you want to. This is not the case in CLI Linux.

In Linux we have a simple command, `rm`, which takes as its main arguments the items to be deleted. It's a single step, and there are in most cases no warnings issued. The simplest cases are `rm myfile.txt` to remove a single file, or to remove a directory `rm -r mydirectory`. `rm` can accept multiple arguments as `rm -r myfile.txt mydirectory`, and it can accept the glob `(*)` as a wildcard.

Let's try this out now. We'll move to our home directory, copy our `killifishGenomes` directory, and then work on removing some things.

```
cd ~
cp -r killifishGenomes killifishGenomesCopy
rm killifishGenomesCopy/scripts/assembly/assemblyV2.sh
rm -r killifishGenomesCopy/scripts/assembly_flye
rm -r killifishGenomesCopy/results/*fasta
```

After removing these files and one directory the structure should look like this:

```
killifishGenomesCopy/
  rawdata
    sample1.fastq.gz
    sample2.fastq.gz
    sample3.fastq.gz
    sample4.fastq.gz
    sample5.fastq.gz
  README.md
  results
  scripts
    assembly
      assembly.sh
    quality_control
      QC.sh
```

5 directories, 8 files

SERIOUSLY, READ THIS.

Don't execute any code in this box

Because `rm` can accept multiple arguments and the glob as a wildcard, certain kinds of typos can be **very damaging**. You may want to remove all the files in a given directory like this:

```
rm /path/to/garbage/*
```

But if you mistakenly type this:

```
rm /path/to/garbage/ *
```

`rm` will refuse to remove directory `garbage` and give you an error (because you didn't supply `-r`) and then go on to **remove every single file in your current working directory** because it interprets the wildcard as a second independent argument.

If you accidentally type:

```
rm / path/to/garbage/*
```

then `rm` may or may not be able to remove the files in `path/to/garbage`, depending on if the current working directory contains the path, but the single `/` means `rm` **will try to delete every file in the root directory**. This would be **very bad**.

Supplying `-r` amplifies the damage done by these typos.

Everyone who has been working at the command line long enough has a horrible story about making a mistake with `rm`, so be very cautious when using it.

2.5 Ownership and Permissions

A pervasive feature of Linux, and one that causes many beginners headaches is the permission system. Most common operating systems that manage multiple users (i.e. Windows and MacOS) manage user access to files and directories invisibly, creating walled off areas of the file system for different users.

Things work a bit differently in Linux. Permissions for files and directories are set explicitly on a case by case basis. Earlier we saw a permission string when we did `ls -l`. When I do this I see

```
drwxr-xr-x  5 nreid cbc          2.0K Dec 18 14:27 killifishGenomes
```

The first field in this output (`drwxr-xr-x`) is the permission string. It is always 10 characters. The first letter, `d` is the file type. It would be `-` for regular files and `d` for directories. The following letters come in groups of three. The groups of three correspond to the permission types for three sets of users: the user owner (`u`) of the file, the user group (`g`) the file is assigned to, and every other user on the system (`o`). The permission types are, in order, “read” (`r`), “write” (`w`), “execute” (`x`). If a given permission is *not granted* to a given set of users, that character will be `-` instead of `r`, `w`, or `x`.

Remember that fields 3 and 4 are the user owner of the file, and the user group the file is assigned to.

So the permission string above indicates that `killifishGenomes` is a directory. The user who owns the directory, `nreid`, has read/write/execute permission (characters 2-4: `rw``x`). The user group `cbc` has read and execute permission (characters 5-7: `r``-``x`) and the rest of the users on the system also have read and execute permission (characters 8-10: `r``-``x`).

A few notes about this:

1. This is a directory, so execute permission may seem somewhat nonsensical. You can execute a program (or a script), but not a directory. In this case it allows you access to see the list of files in the directory and their metadata. If you don't have execute permission, you can't do anything inside the directory.
2. This directory is inside my home directory, where I am the only user with any permissions at all, therefore, per point one above, even though other users nominally have read and execute access, they effectively do not.

2.5.1 Changing permissions

We use `chmod` to change permissions. There are two ways to change permissions. We're going to learn the easy, more verbose one. To give yourself (u), a user group (g) or all other users (o) read, write, or execute access to a file or directory, you can specify one or more of these sets as a string, and then add +, or remove - one or more permission types as a string. For example, to give everyone full access, you can do `chmod ugo+rxw filename`. To take away write access from everyone (locking up raw data like this is always a good idea so nobody accidentally deletes it) `chmod ugo-w filename`. To apply a permission string recursively to all files and subdirectories in a directory, you can simply add -R as in `chmod -R ugo+rxw mydirectory`.

Try this now: Take away all permissions for the group and the rest of the system users on the directory `killifishGenomes` and all its subdirectories.

```
chmod -R go-rwx killifishGenomes
```

If you do `ls -l` you should see:

```
drwx----- 5 nreid cbc          2.5K Apr  4 16:51 killifishGenomes
```

And for `ls -l killifishGenomes/*`:

```
-rw----- 1 nreid cbc          67 Dec 18 14:27 killifishGenomes/README.md
-rw----- 1 nreid wegrzynlab    5 Apr  4 16:51 killifishGenomes/test.txt
```

`killifishGenomes/rawdata`:

total 20

```
-rw----- 1 nreid cbc 0 Dec 18 14:27 sample1.fastq.gz
-rw----- 1 nreid cbc 0 Dec 18 14:27 sample2.fastq.gz
-rw----- 1 nreid cbc 0 Dec 18 14:27 sample3.fastq.gz
-rw----- 1 nreid cbc 0 Dec 18 14:27 sample4.fastq.gz
-rw----- 1 nreid cbc 0 Dec 18 14:27 sample5.fastq.gz
```

```
killifishGenomes/results:
total 20
-rw----- 1 nreid cbc 0 Dec 18 14:27 sample1.fasta
-rw----- 1 nreid cbc 0 Dec 18 14:27 sample2.fasta
-rw----- 1 nreid cbc 0 Dec 18 14:27 sample3.fasta
-rw----- 1 nreid cbc 0 Dec 18 14:27 sample4.fasta
-rw----- 1 nreid cbc 0 Dec 18 14:27 sample5.fasta

killifishGenomes/scripts:
total 12
drwx----- 2 nreid cbc 1024 Mar  5 11:06 assembly
drwx----- 2 nreid cbc 1024 Mar  5 11:07 assembly_flye
drwx----- 2 nreid cbc  512 Mar  5 10:59 quality_control
```

2.5.2 Changing groups

For any file you own, you can change the group to any user group you are a member of using `chgrp`. To first see the groups you are a member of type `groups username` (but use your username). For example, I am a member of `reidlab` and `cbc`. I can change this directory (and all its contents recursively) from `cbc` to `reidlab` with `chgrp -R reidlab killifishGenomes`.

2.6 Getting Help

Perhaps you're getting the impression that working at the command line is going to require mastery of many, many small details. To a degree this is true. Fortunately, there are lots of ways to get help.

2.6.1 Built-in help

The most basic way to get help with a program is to try to get the program itself to print its *usage*. That is, you can ask the program to write how it should be used, and a brief description of its command line arguments. One or more of the following approaches will usually work:

1. Entering the command with no options: `<command>`
2. Entering the command with the flag `-h`: `<command> -h`
3. Entering the command with the flag `--help`: `<command> --help`

Most programs will likely print the usage with one or more of these approaches.

Another form of built-in help are manual, or `man` pages. Not every program has a `man` page, but most programs included as part of a Linux distribution will. To access a `man` page, enter

`man <command>`. The documentation will display interactively. To exit the man page and return to the shell prompt, type `q`.

Try these approaches for the commands `find` and `cat`. **Note:** `cat` with no options will drop you into an interactive session with `cat`. To kill the process and get back to the prompt, try typing `ctrl-.` or `cmd-x`.

2.6.2 The Internet!

There are **tons** of resources available on the internet for learning Linux and the bash shell, troubleshooting particular commands, and accomplishing specific common tasks. Good places to look are:

1. [The bash manual](#)
2. [Introduction to Linux](#)
3. [stack overflow](#) - a community where users can ask and answer questions on a variety of topics.
4. [unix stack exchange](#)

But there are many sites, ranging in quality and focus that can help you figure out answers to questions. Even advanced users regularly consult pages like this and search google for help.

2.6.3 AI/large language models

Since 2022, the emergence of large language models like [chatGPT](#) has been transforming how we do work in bioinformatics. These models allow users to ask questions and receive answers in natural language, and they are remarkably adept at producing and interpreting code, at least for relatively straightforward tasks. They can dramatically increase the speed at which experienced users can produce code, and help elevate the abilities of new users.

For a simple example of how you can get help, visit [chat.openai.com](#) and try this question: “In bash, how can I use the `find` command to find all files created since 2022?”

Warning

We encourage the use of LLMs as an aid in this program, but caution students that they can sometimes do inefficient, unpredictable or incorrect things, and that they should understand what code is doing, and verify that it is correct by checking documentation and validating it through testing. Also, be aware that sometimes an LLM will produce a result that is technically correct, but problematic for other reasons.

For example, if you ask ChatGPT for code to calculate the variance from a sample, it will give you code for a standard statistics textbook equation for estimating variance. This

method is subject to [numerical instability](#), however, and will yield inaccurate estimates under some circumstances. If you know enough to *ask* ChatGPT about the numerical stability issue, it will immediately suggest fixes, but we don't always know what we don't know.

2.7 Asking questions about the system

What is the name of the system you are on now? Try `hostname`. On a computer cluster like Xanadu this is really important. There are many different nodes. Sometimes a job will fail as a result of a problem with the system rather than some kind of user error. It's good to know when troubleshooting which one you were using.

Which Linux distribution and kernel version is your system running? Try `hostnamectl` (or `hostinfo` on a local MacOS machine).

How much memory is available on this system? Try `lsmem` or `cat /proc/meminfo` for a detailed view. You'll note that as you're likely on a login node, the memory available will not be impressive (something like 8 gigabytes).

How many and which CPUs are on this system? Try `lscpu` or `cat /proc/cpuinfo`. From `lscpu`, the total number of CPUs (or cores) on the system is the number of sockets times the number of cores per socket. On the login node that will probably be something paltry like 8.

What processes are currently running on the system? Try `top`. This is a list of running processes. To sort by CPU usage, type `shift-p`. To sort by memory usage type `shift-m`. Since you're most likely on a login node (rather than a heavy duty compute node), hopefully you will not see any processes using a substantial percentage of CPU or memory. Type `q` to quit. You can also use `ps aux`.

How long and how intensively has the system been used? Try `uptime`. Per the man page, `uptime` will tell you how long the system has been running, how many users are currently logged on, and the system load averages for the past 1, 5, and 15 minutes. Note that "Load averages are not normalized for the number of CPUs in a system, so a load average of 1 means a single CPU system is loaded all the time while on a 4 CPU system it means it was idle 75% of the time."

How full are available storage systems? Try `df -h`. Xanadu has several network-attached file systems (NFS). Hopefully none are too close to full..

How much space is a given directory using? We've seen that `ls -l` can tell us how much disk space a file uses, but it won't sum everything up for a whole directory. Try `du -sh`. This will give you an accounting of the space used in your current working directory. To tally

up each directory in the current working directory individually, try `du -sh *` or for a specific directory `du -sh killifishGenomes`.

Which other users are logged in to this system?: Try `who`. Xanadu has many login and compute nodes. This list is only users who are currently logged in (though they may be idle).

Which groups does a user belong to? Try `groups <username>`. Try your own username

Which users belong to a group? Try `getent group <groupname>` (try `cbc` for Computational Biology Core and associated faculty and staff).

2.8 Basic Linux Commands

Command	Description
<code>pwd</code>	print working directory
<code>cd</code>	navigate through directories
<code>ls</code>	list directories
<code>find</code>	search for files
<code>echo</code>	print arguments to standard out.
<code>mkdir</code>	create directory
<code>mv</code>	rename or move files
<code>cp</code>	copy files or directories
<code>touch</code>	create empty file
<code>cat</code>	display file contents or concatenate
<code>rm</code>	delete files or directories
<code>man</code>	get help
<code>less</code>	display paged outputs
<code>chmod</code>	change file permissions
<code>wget</code>	download files from the internet

2.9 Exercises

2.9.1 Practical exercise:

1. Using the commands we covered above, start a fake project directory. It should have exactly this structure:

```

fakeProject
  data
    Sample1.R1.fq.gz
    Sample1.R2.fq.gz
    Sample2.R1.fq.gz
    Sample2.R2.fq.gz
    Sample3.R1.fq.gz
    Sample3.R2.fq.gz
    Sample4.R1.fq.gz
    Sample4.R2.fq.gz
    Sample5.R1.fq.gz
    Sample5.R2.fq.gz
  genome
    mygenome.fa
  results
    alignments
      Sample1.bam
      Sample1.bam.bai
      Sample2.bam
      Sample2.bam.bai
      Sample3.bam
      Sample3.bam.bai
      Sample4.bam
      Sample4.bam.bai
      Sample5.bam
      Sample5.bam.bai
  scripts
    alignment.sh

```

6 directories, 22 files

Names ending in `.bam`, `.bam.bai` or `.fq.gz` are files (these are suffixes corresponding to common data formats, but for now create empty files with these names). Remember, you can use shell expansions, `touch` and `mkdir`. To do this quickly. If you mess up, you should know how to remove it all and start over.

2. Create a directory named after your username here `/core/cbc/ISG5311/2024` (but use the current year if it is no longer 2024!) and move or copy this directory there.
3. Assume you want other users in your user group, but no other users on the cluster to have access to your **data**, but nothing else. Change the permissions accordingly.

2.9.2 Quiz: See Blackboard Ultra for this section's quiz.

3 Working with Files

Learning Objectives:

Move files between computers

Work with file compression

Inspect files

Use regular expressions with `sed` and `grep`

Pipe inputs and outputs

In the previous section, we learned about the shell, how to navigate the file system and how to manipulate files and directories within the file system. Now we're going to get into how we can transfer files on and off the server, edit, inspect and summarize files in the shell. These are essential skills that will give you the flexibility to monitor your analyses and check input and output files against your basic expectations.

3.1 Moving files between computers

You will quite frequently need to move files on and off a remote computer cluster. We will sometimes use specialized pieces of software for retrieving data from particular public databases, but there a number of pieces of widely available software it's good to be familiar with. Here we'll discuss:

1. `wget` and `curl`
2. `scp`
3. `rsync`
4. GUI-based FTP clients and Globus

3.1.1 `wget` and `curl`

These are two commonly used utilities for downloading files via HTTP and FTP protocols. If you've got a URL pointing to a file you want, either of these will do. Both have lots of features and support several file transfer protocols, but for the most common use cases we'll encounter, simply invoking the command and providing the URL as the first argument will work will start the download. For `wget` you don't even need to specify a name for the downloaded file, but

for `curl` you'll need to redirect the output to a file. **Try one of the following** to download the *Fundulus heteroclitus* genome from the ENSEMBL database:

```
wget ftp://ftp.ensembl.org/pub/release-105/fasta/fundulus_heteroclitus/dna/Fundulus_heteroclitus-3.0.2.dna.tot
curl ftp://ftp.ensembl.org/pub/release-105/fasta/fundulus_heteroclitus/dna/Fundulus_heteroclitus-3.0.2.dna.tot
```

3.1.2 scp

`scp` is a *secure* version of the `cp` command we learned in the previous chapter. We can use this to securely, i.e. in encrypted fashion, transfer data between computers. The usage is the same as `cp` (requiring the `-r` flag to copy a directory), except that with `scp` you also need to provide an address for a remote host in front of the path like this: `username@remotehostaddress:/path/to/target/file.txt`. You can copy data to or from a remote host. You can use `.` if you want the file to simply transfer to the current working directory.

On the Xanadu cluster, you cannot use the normal login nodes (i.e. `xanadu-submit-ext.cam.uchc.edu`) to transfer files. A special transfer host has been set up for this purpose. The address for that host is `transfer.cam.uchc.edu`. **Try this now:** Copy the *F. heteroclitus* genome we just downloaded from Xanadu to your local computer. Open a terminal window and *do not* connect to Xanadu. Enter the following command (be sure to edit the command to reflect your username and the path containing the genome on Xanadu):

```
scp nreid@transfer.cam.uchc.edu:~/Fundulus_heteroclitus.Fundulus_heteroclitus-3.0.2.dna.tot .
```

Hopefully it is self-evident how to transfer files **to** the remote host with this method. You can feel free to delete the transferred file, as we will not be continuing to use it.

3.1.3 rsync

`rsync` is like a much more elaborate version of `scp`. While its usage is generally much more flexible, the key improvement over `scp` is that `rsync` can be used to “synchronize” directories across machines. That is, if you have a copy of some analysis results on your local machine and then you update some part of the analysis on the remote cluster, if you tell `rsync` to copy the remote results directory to your local computer, it will check the individual files to see if they’ve changed and skip them if they haven’t. This is a pretty useful feature. This also means it can resume failed transfers without having to start from scratch.

The usage of `rsync` is similar to `scp`, except that when transferring a directory we recommend using the flags `a v` and `z`. To move your `killifishGenomes` directory from Xanadu to your local machine you can try: `rsync -avz nreid@transfer.cam.uchc.edu:~/killifishGenomes .`

taking care to edit your username and ensuring the path on Xanadu to your directory is correct. This will transfer the entire directory.

`rsync` has lots of options, use `man rsync` to read up on it.

i Note

A subtle but important detail: if you include a trailing `/` on the source directory like this: `rsync -avz nreid@transfer.cam.uchc.edu:~/killifishGenomes/ .` Then the **contents** of the source directory will be transferred to the target directory, but not the enveloping directory.

3.1.4 FTP clients and Globus

The above options for transferring files are all command-line programs available on most, if not all, Linux distributions. Sometimes you may want to use other methods to move data. Some users employ GUI programs that use the file transfer protocol (FTP), such as [FileZilla](#). You can use the transfer node `transfer.cam.uchc.edu` and your Xanadu credentials to move files this way.

When you have very large amounts of data to transfer, a more secure and fault tolerant program can be helpful. On Xanadu we use Globus, a platform available on many institutional HPCs for moving data. We won't cover it right now, but you can see a guide to getting started using it on Xanadu [here](#)

3.2 File compression

'omic data files are often very large. A single copy of a human genome is around 3.3 gigabytes. A raw, unassembled sequence of a human genome at 30x coverage might be 180 gigabytes. Data at this scale is cumbersome to transfer and analyze, and expensive to store. Compression eases this burden somewhat, making these files 1/3 to 1/4 the size. So wherever we can, we try to keep data files compressed, and use tools that can read and write data in compressed form. There are a few means of compression that we'll encounter during this course, but here we'll cover `gzip`.

`gzip` refers to both a compression algorithm **and** the most common piece of software used to implement it. Above we download a genome for *F. heteroclitus*. The file name is `Fundulus_heteroclitus.Fundulus_heteroclitus-3.0.2.dna.toplevel.fa.gz`. The `.gz` suffix is commonly applied to indicate that a file has been `gzip`-compressed. If you list the contents of the directory containing this file with `ls -lh` you should see this compressed genome file is 278 megabytes:


```
-rw-r--r--  1 nreid cbc   278M Mar  6 19:32 Fundulus_heteroclitus.Fundulus_heteroclitus-3.
```

Try this now: Decompressing the file with

```
gunzip Fundulus_heteroclitus.Fundulus_heteroclitus-3.0.2.dna.toplevel.fa.gz
```

You'll see that `gunzip` removes the `.gz` suffix, and `ls -lh` indicates the file is now 992 megabytes, giving a compression ratio of 3.57.

```
-rw-r--r--  1 nreid cbc   992M Mar  6 19:32 Fundulus_heteroclitus.Fundulus_heteroclitus-3.
```

You can compress the file again with:

```
gzip Fundulus_heteroclitus.Fundulus_heteroclitus-3.0.2.dna.toplevel.fa
```

Note that compressing the file takes longer than decompressing.

! Important

Remember, because many, if not most tools that process genomic data can read and write compressed files, unless you know you need uncompressed data, **make sure you keep files compressed.**

You are likely to encounter a particular type of gzipped file known as a **Tape ARchive** (suffix `.tar`) or informally as a **tarball**. Tarballs are entire directories of files that have been put into an archive file and then gzipped (suffix `.tar.gz`). You will commonly encounter source code for uncompiled software distributed as tarballs, and sometimes large sets of raw data. To decompress them you can use the utility `tar` like this:

```
tar -xvzf mydir.tar.gz
```

The `-xvzf` flags indicate you want to extract the tarball (`-x -f`) that the tarball is gzipped (`-z`) and that you want it to print out what it's doing as it goes (`-v`, for *verbose*, a common option with command-line software). See the man page for more details.

3.3 Basic file inspection

Linux has a number of simple tools we can use to inspect uncompressed files in plain text. Let's download some files we can do some exploring with. To keep it simple we'll download a catalog of public domain books available through Project Gutenberg, and a plain text copy of

Charles Darwin’s “On the Origin of Species By Means of Natural Selection”. Let’s also rename the file to something more descriptive.

```
# Catalog
wget https://www.gutenberg.org/cache/epub/feeds/pg_catalog.csv
# Origin of Species
wget https://www.gutenberg.org/ebooks/1228.txt.utf-8
mv 1228.txt.utf-8 darwin1859.txt
```

3.3.1 less

An easy way to scroll through this file is with the program `less`.

```
less darwin1859.txt
```

Pressing the up or down arrows will scroll through the text. The space bar will move one screen’s worth of text at a time. Press `q` to exit. `less` can also view gzip-compressed text files without decompressing them.

3.3.2 head, tail and cat

We can print lines from the file to the screen using three commands.

`head` prints the *beginning* of the file. It prints the first 10 lines by default, but the flag `-n` can modify this:

```
head -n 20 darwin1859.txt
```

`tail` prints the the *end* of the file. It also prints 10 lines by default and accepts the `-n` flag.

```
tail -n 20 darwin1859.txt
```

The `-n` flag in `tail` can also accept a number in the format `+20` to indicate it should print the end of the file *starting at line 20*. Be careful with this one if the file is large.

```
tail -n +20 darwin1859.txt
```

To cancel the process, remember, try `ctrl-x` or `command-..`

To print out the *entire file* to the terminal you can use `cat`.

```
cat darwin1859.txt
```

3.3.3 Counting with `wc`

To count the number of lines, words and bytes (a proxy for characters), you can use `wc`.

```
wc darwin1859.txt
```

You can use `wc -l` to output only the number of lines.

3.3.4 `cut`

With tabular data (i.e. our catalog of books) you can cut out columns with `cut`. Try inspecting the file with the above commands first. With `cut`, you can extract a column (or field) with the flag `-f`. You'll see that each of the fields in this table is separated by a `,`. `cut` expects a tab character by default, so we must also specify a delimiter with `-d`. To extract column 3, the publication date, we would do:

```
cut -f 3 -d "," pg_catalog.csv
```

You'll likely notice some messiness. `cut` is not smart, and doesn't recognize that the commas in a title like "A Medium of Inter-communication for Literary Men, Artists, Antiquaries, Genealogists" are not meant to be field separators.

3.4 Regular expressions and `grep`

We very frequently want to search a text file for occurrences of some pattern, perhaps a word, phrase, or series of numbers. `grep` is a frequently used for this purpose. Provided a file and a matching pattern, it's default behavior is to return all lines in the file containing the matching pattern. It has a lot of flexibility, however. You can have it *exclude* all lines with matches, return lines preceding and succeeding matches, print line numbers, just a count of occurrences, or instead of the whole line, only the matching string. You can check the `man` page for more detail.

How do we specify these matching patterns? with **regular expressions** (sometimes shortened to **regex**). You can think of regular expressions as a very elaborate expansion of the `*` wildcard we have already seen (e.g. `*.txt` to match any file with suffix `.txt`). [This](#) is a great page explaining lots of regex features, and is worth skimming through.

To introduce this, we will work through some examples with `grep`. **Follow along in the terminal** using the file `darwin1859.txt` that we downloaded above.

We'll start with the simplest regex, an unambiguous text string, by searching for every occurrence of the word `species`:

```
grep "species" darwin1859.txt
```

If we want only a count of these occurrences, we can do:

```
grep -c "species" darwin1859.txt
```

In these cases, “species” is our regular expression. What if we want to introduce some ambiguity? In regex syntax `.` is the wildcard character. It matches a single character without respect to identity (including whitespace characters). We could find “species” without regard to capitalization by:

```
grep ".pecies" darwin1859.txt
```

Note that if you actually wanted to match a period character, you need to *escape* it with a `\`. Escaping tells the regex interpreter to treat an otherwise special character literally. So if you want to match cases where species is the last word in a sentence:

```
grep ".pecies\" darwin1859.txt
```

Of course, this only works when the rest of the word can’t have any other unintended matches. Searching for “had” with `.ad`, would not only match “Had” and “had”, but every other three letter string ending in “ad”. Try it now:

```
grep ".ad" darwin1859.txt
```

`grep` has an option `-w` that will match only “words”, i.e. only matches that are preceeded or followed by “non-word” characters or line starts/endings.

```
grep -w ".ad" darwin1859.txt
```

But this will also return words like “bad”, “mad”, etc. In regex syntax we can specify groups of characters, or ranges of characters inside square brackets to be more specific.

```
grep -w "[Hh]ad" darwin1859.txt
```

This will match only the full words “Had” and “had”.

We can *exclude* characters by adding a `^`. Had/had are very common. If we only want to match other words we can try:

```
grep -w "[^Hh]ad" darwin1859.txt
```

What if there is some ambiguity in the length of the pattern we wish to match? There are a couple options here. Say we wish to see all words ending in “ing”? We can use the asterisk `*` to say, *match 0 or more of the preceding character* along with a bracketed set of letters. In this case we will quit using `-w` as a crutch and be explicit about the match pattern by excluding any lower-case letter characters following the match.

```
grep "[A-Za-z]*ing[~a-z]" darwin1859.txt
```

There are multiple “flavors” of regular expressions. We’ve been using the basic version. There are extensions with more features. We can enable extended regexes in `grep` with `-E`, and perl-style regexes with `-P`. For example, we can specify a number or range of repetitions with `{1,3}` for *match 1-3* of the preceding character:

```
grep -E "[^A-Za-z][A-Za-z]{1,3}ing[~a-z]" darwin1859.txt
```

With perl-style regexes (`-P`) we can also use “lookarounds”. These divide a regex into two pieces, a core matching sequence, and an adjacent matching sequence that is required (or excluded), but not returned as part of the total match. We can match words following the word species with a “positive lookbehind” like this:

```
grep -P "(?<=[Ss]pecies )[A-Za-z]+" darwin1859.txt
```

Since in these last couple cases, we don’t actually know what our matches are going to be, it can be helpful to return *only* the matches. You can use `-o` for this:

```
grep -o -P "(?<=[Ss]pecies )[A-Za-z]+" darwin1859.txt
```

i Note

When you construct regular expressions, you want to test them carefully to see that they match what you want, and **only** what you want. It’s pretty easy to come up with something you will work, only to discover later there are lots of unintended matches. On this note, large language models like ChatGPT can be pretty effective at writing and interpreting regexes for you, but they can make mistakes, and they can’t necessarily anticipate unintended matches in your data any better than you can, so you should still test them before relying on them heavily in code.

3.5 STDIN, STDOUT, STDERR, redirection and the pipe

We have now seen a few different tools we can use to inspect and summarize files. Though each alone is fairly limited, one of the strengths of the Linux command-line environment is the

ability to easily chain simple tools like these together in powerful ways. To understand how this works, we need to cover a few concepts.

Linux programs operate on three main streams of data which users can redirect in various ways. First is the **standard input** (sometimes abbreviated STDIN). Programs can often read data from this stream. The second is the **standard output** (STDOUT) programs often write output to this stream. The third is the **standard error** (STDERR). This stream is often used by programs to send warning or error messages (though scientific software written by scientists, rather than software engineers, will sometimes write output directly to a file, and warnings or errors to stdout, as you will see later in the course). These three streams have numerical file descriptors 0, 1, and 2, for stdin, stdout and stderr.

For programs that write output to stdout and errors to stderr by default, these outputs will be written to the terminal. We saw this above with `grep`. When we searched our text file, the lines we requested streamed across our terminal window. If we made any typos, say referring to a non-existent file such as `darwin1959.txt`, the resulting error message would have come from the stderr stream.

We can **redirect** these streams, however, causing them to be written to files, or if we wish, taking the stdout stream from one program and hooking it up to the stdin stream for another.

Try the following:

To redirect the stdout stream to a file, let's modify one of our commands above with `>`.

```
grep "species" darwin1859.txt >species_lines.txt
```

We have now created a new file with every line containing the string “species” (we can append to an existing file with `>>`).

As above, if we did something that caused an error, such as referring to a file that doesn't exist, we would see an error in the terminal:

```
grep "species" darwin2000.txt >species_lines.txt
```

That error was written to stderr, and we can redirect it to a file like this:

```
grep "species" darwin2000.txt >species_lines.txt 2>species_lines.err
```

`2>` is taking the stderr (which has file descriptor 2) and sending it to a file. The command creates two files. Though the program failed with an error, the redirect still created the file `species_lines.txt`. It's important to know this is the usual behavior. Seeing an output file does not mean a program executed successfully. The second file, `species_lines.err` now contains the output from stderr.

You can even redirect stderr to stdout with:

```
grep "species" darwin2000.txt 2>&1
```

The `&1` refers to the stdout file descriptor. If you just wrote `2>1` stderr would be written to a new file called `1`.

3.5.1 The pipe

The character `|` is usually referred to as the **pipe**. It allows us to redirect the stdout from one program to the stdin for another, thus *pip*ing together one or more programs, or creating a *pipeline*, though the term pipeline is also used in a much broader sense to refer to any workflow consisting of many sequential steps.

Above we extracted every line containing the word `species`. We could have used `-c` to count lines. We can also *pipe* the output to `wc` to count the number of lines:

```
grep ".pecies" darwin1859.txt | wc -l
```

Here the pipe is sending the stdout from `grep` to the stdin for `wc`.

For something a little less trivial, let's see a really common linux idiom:

```
grep -o -P "(?<=[Ss]pecies )[A-Za-z]+" darwin1859.txt | sort | uniq -c
```

The `grep` command is extracting every word that comes after “species”. We then use two commands piped together, `sort` and `uniq` to tally up their frequencies. `uniq` emits unique lines, but it can only identify duplicates if they are adjacent to each other, so first we must sort them. `-c` tells `uniq` to also write the counts of each element. We can sort this yet again numerically to see the most common words following “species”:

```
grep -o -P "(?<=[Ss]pecies )[A-Za-z]+" darwin1859.txt | sort | uniq -c | sort -g
```

Let's try tallying up word frequencies for the entire document. Instead of using `-w`, let's use `\b` to indicate word boundaries, restrict ourselves to words with 4 or more characters (`{4,}`), and let's only look at the 50 most common (`tail -n 50`):

```
grep -o -P "\b[A-Za-z]{4,}\b" darwin1859.txt | sort | uniq -c | sort -g | tail -n 50
```

This isn't totally optimal as it is case-sensitive, but hopefully you can begin to see how these tools will let you rapidly inspect and summarize files in useful ways.

3.6 sed

Above we’ve used regexes and **grep** to match text. We can also edit text on the fly using regexes with **sed** (which stands for **Stream EDitor**). **sed** has lots and lots of features. See extensive documentation [here](#) and a more in-depth tutorial [here](#). We’re just going to cover two features here. Replacement operations, and controlling which lines are printed.

3.6.1 Replacements

sed’s replacement operator has a relatively simple syntax. `s/matchpattern/replacement/`. The **s** indicates a replacement operation, and the `/`’s delimit the match and replacement strings. Any character can be used as a delimiter, e.g. `s,matchpattern,replacement,.`

The matching patterns use regular expressions as in **grep** and extended regexes can be used with `-r`, but perl-style regexes (i.e. those with lookarounds) are not available.

Try this now: Imagine we would like to make our Project Gutenberg catalog a little less formal. Let’s change all occurrences of “Darwin, Charles” to “Darwin, Chuck”. First let’s use **grep** to extract only lines containing our target string:

```
sed 's/Darwin, Charles/Darwin, Chuck/g' pg_catalog.csv
```

sed’s default behavior is to edit only the first match on each line. In this case, the trailing **g** in the matching expression means tells it to edit all occurrences.

You probably noticed that by default, **sed** writes all output to stdout, rather than editing the original file. If you *want* to edit the original file, you can use `-i` for “edit in place”, but destructive editing *en masse* like this is not usually advisable. Let’s add a **grep** command to extract only our target lines to see them more clearly:

```
grep "Darwin, Charles" pg_catalog.csv | sed 's/Darwin, Charles/Darwin, Chuck/g'
```

For cases where we have ambiguity in the match that we wish to preserve in the replacement, we can use capturing groups. These are a feature of extended regexes, and require the `-r` flag. Capturing works by putting parentheses around the target string. Captured strings can then be invoked with `\1`, `\2...` for the first, second string, etc. Let’s find every name formatted as “<SURNAME>, Alfred” and reformat as “Alfred”The Greatest Author of All Time” <SURNAME>“:

```
sed -r 's/([A-Z,a-z,-]*), (Alfred [A-Za-z]*),/\2 "The Greatest Author of All Time" \1,/g'
```

This will substitute names like “Brehm, Alfred Edmund” for ““Alfred Edmund”The Greatest Author of All Time” Brehm”.

3.7 Line printing

We can also use `sed` to print specific lines by number. To print line number 24286, we tell `sed` to print no lines (`-n`) except the ones we specify 24286p.

```
sed -n '24286p' pg_catalog.csv
```

Alternatively, we could print, say, every 4th line, starting at line 2:

```
sed -n '2~4p' pg_catalog.csv
```

3.8 awk

`awk` is a program (and a language) that can be used to extract, process, or reformat data. In combination with `sed`, `grep` and regular expressions, we have a very powerful set of tools for examining, validating, and/or reformatting data quickly without requiring specialized software (although sometimes we want to use specialized software instead of reinventing our own rickety wheels in custom bash scripts). We'll cover `awk` superficially here, but see [here](#) for the documentation, and [here](#) for a shorter tutorial.

Let's use `awk` to do a little processing of a genome annotation file in GTF format. First download and decompress it:

```
wget https://ftp.ensembl.org/pub/release-111/gtf/fundulus_heteroclitus/Fundulus_heteroclitus-3.0.2.111.gtf.gz
gunzip Fundulus_heteroclitus.Fundulus_heteroclitus-3.0.2.111.gtf.gz
```

We'll get into genome annotation files more later, but for now you should know GTF is a tab-delimited format that contains the locations of important genomic features. For each feature, column 1 gives the sequence (often a whole chromosome, sometimes a chromosomal fragment) that the feature is found on, columns 4 and 5 give the start and end positions (1-based, inclusive) of the feature, and column 3 gives the type of feature (exon, CDS, transcript, gene). Features are hierarchical, so coding sequences (CDS) and exons always have a transcript parent feature, and transcripts always have a gene parent feature. In a GTF, all these identifiers are supposed to be provided for each record, though gene annotation file formats are very frequently violated by different pieces of software. The transcript and gene identifiers are found as part of a long semicolon-separated list in column 9.

Let's first use `awk` in the simplest possible way. We'll print only lines containing gene features. `awk` automatically parses text files with white space as field separators, so we can access the columns (or fields) with special variables `$1`, `$2`, `$3...` with `$0` referring to the entire line. We can match individual fields with a regular expression like this:

```
awk -F "\t" '$3 ~ /transcript/' Fundulus_heteroclitus.Fundulus_heteroclitus-3.0.2.111.gtf
```

Here we're using `-F "\t"` to tell `awk` to use only tabs as field separators (to the exclusion of spaces). Piping this whole thing to `wc -l` we would see this file contains 35,597 transcript records.

We can use logical statements to require multiple matches to pull out transcripts matching a gene identifier:

```
awk -F "\t" '$3 ~ /transcript/ && $9 ~ /ENSFHEG00000021448/' Fundulus_heteroclitus.Fundulus_heteroclitus-3.0.2.111.gtf
```

`awk` has considerably more utility than this, however. I often use it to reformat files.

Another common format used to store information about genomic regions is `BED`. `BED` has only 3 required columns: the sequence identifier, the start position and the end position. The trick here is that whereas `GTF` is 1-based and fully closed, `BED` is 0-based and half-open. This means that in a `GTF` file, an interval of 1-1000 refers to the first 1000 bases, including the start and end point of the interval. In `BED` format, however, the first base in a sequence is numbered 0, and the end base in an interval is not included as part of the interval, making the `BED` interval 0-1000. See this [blog](#) for a discussion.

If you find this fully closed vs half-open, 0-based vs 1-based stuff irritating, know that it will come up repeatedly, as it has for many decades (see [this](#) 40+ year old polemic)

At any rate... we can easily output our `GTF` intervals in `BED` format like this:

```
awk -F "\t" '{OFS="\t"}{newstart=$4-1}{print $1,newstart,$5}' Fundulus_heteroclitus.Fundulus_heteroclitus-3.0.2.111.gtf
```

Here we have grouped commands with `{}` and specified an “output field separator” (`OFS="\t"`) because `awk` will by default write spaces instead of tabs. We told `awk` to print field 1, our new variable `newstart`, and field 5.

Note that in this case, `GTF` has a few header lines (beginning with `#`) that we have mangled and included.

We can also summarize our data by doing operations across lines. Let's figure out how much of our genome falls into annotated genes:

```
awk -F "\t" '{if($3 ~ /gene/) n+=$5-$4+1 } END {print n}' Fundulus_heteroclitus.Fundulus_heteroclitus-3.0.2.111.gtf
```

In this case, we use a conditional `if($3 ~ /gene/)` to say that if field 3 is a gene record, we should add the difference between fields 5 and 4 plus 1 to the variable `n`. `n+=...` is a shorthand for `n=n+...`, so for each subsequent record the value of `n` is incremented rather than replaced by a new value. In `awk`, `END` indicates the following command should be executed when the file has finished processing.

We see here that 474,790,822 bases of our 1 gigabase genome falls inside of gene annotations. To figure out how much is actually exonic is a little more complicated and difficult to achieve with awk alone because each gene may have multiple transcripts, each with their own overlapping exons, though it could be done.

3.9 Linux Commands in This Section

Command	Description
cmnd	desc

3.10 Exercises

3.10.1 Quiz:

This week's quiz is a practical demonstration of the skills from this section. Find it on Xanadu here: `/core/cbc/ISG5311/quizzes/week2quiz.docx`. Transfer it to your local computer, and using Microsoft Word, answer the questions it and upload it to blackboard. <details...>

References

Part III

HPC and SLURM

Part IV

Learning Objectives

Learning Objectives:

Edit files

Use environment variables

Write loops to efficiently do repetitive tasks

Edit files on Xanadu in **nano**

Connect Visual Studio Code to Xanadu to edit files

Connect your desktop to Xanadu's file system to access files

Write and execute scripts

Query SLURM to determine resource availability

Submit work and a resource request to be managed by SLURM

Monitor the progress of work on SLURM

Evaluate the results of work submitted to SLURM

Start an interactive session on a compute node

Resources:

4 Scripts and putting it all together

Learning Objectives:

Edit files

Use environment variables

Write loops to efficiently do repetitive tasks

Edit files on Xanadu in **nano**

Connect Visual Studio Code to Xanadu to edit files

Connect your desktop to Xanadu's file system to access files

Write and execute scripts

We've now gotten some experience navigating the Linux file system, and inspecting and summarizing files. We've seen how we can use pipes to connect simple programs together to quickly get insight into our data. These are key ingredients to building up to analysis of data that you can be confident in. We're now going to introduce some concepts that will help us write **scripts**, which are essentially shell commands organized into a file and executed as a batch, rather than executed interactively.

scripting is not categorically different from *programming*. The terms are generally used for polar ends of a spectrum of complexity. If you, for example, write bash code to download some data from a database, decompress it, and set permissions, most people would refer to that as a scripting. In contrast, if you write code in c++ to parse thousands of sequence alignments and use complex numerical algorithms to estimate some statistical quantity from them, most people will refer to that as *programming*. In both cases, you are writing instructions for a computer to follow.

In this course we're going to focus mostly on the simpler end of the spectrum: using shell scripts to automate chunks of data analysis performed by other programs written in languages such as **c++**, **python**, or **R**.

4.1 One-Liners

In the previous chapter we introduced the pipe as a way to link the inputs and outputs of multiple shell commands. Short piped commands are sometimes referred to as “one-liners” and you can find lots of useful repositories of them at pages like [this](#) and [this](#). A quick skim

through these, especially after we've moved a little further through the course will be super helpful.

4.2 Environment variables

A key feature of BASH is the use of **environment variables**. These are objects used to store text strings. These strings are typically used in 2 ways: to generalize code in shell scripts, or to configure the system.

4.2.1 Invoking variables

Some variables are set automatically (usually those used to configure the system or shell options). To see a list try the command `env`. You should see a long list, but among them is a pretty straightforward variable: `USER`. To invoke it, try any one of the following:

```
echo $USER
echo ${USER}
echo "${USER}"
```

The key thing here is the `$`. The curly brackets and quotes can slightly modify how the variable itself is invoked or understood by the shell (more in a moment), but the `$` is how you let the shell know you're calling out a variable.

Variables can be interpolated into text strings, or commands in a variety of ways. Try this, for example:

```
echo "My username is $USER"
```

4.2.2 Setting variables

Users can also create variables, or modify existing ones. To create a variable holding a path, for example:

```
MYHOMEDIR="/home/FCAM/$USER/"
echo $MYHOMEDIR
```

In this case you can see we've defined a variable using another variable. We can equally well define output files this way:

```
MYOUTFILE="/home/FCAM/$USER/${USER}_testfile.txt"
echo $MYOUTFILE
echo "hello world!" >$MYOUTFILE
cat $MYOUTFILE
```

4.2.3 An aside about quotes in BASH

Now that we're getting a little more into bash, we need to be careful when thinking about quotes and whitespace.

Try to assign the text string “Hello World” without quotes like this:

```
HW=Hello World
echo $HW
```

You should get the error `-bash: World: command not found`. This is because bash tried to assign the string “Hello” to variable “HW”, then saw the whitespace and treated “World” as a separate command, which happens not to exist. The entire command failed.

If we wrap the text in double quotes it will work:

```
HW="Hello World"
echo $HW
```

If we wrap the text in single quotes it will work:

```
HW='Hello World'
echo $HW
```

But what's the difference between double and single quotes? Single quotes prevent variable interpolation, leading to *very* different results:

```
HWD="Hello $USER"
echo $HWD
HWS='Hello $USER'
echo $HWS
```

Quoting generally ensures whitespace is maintained as part of a string, which is sometimes very important. Single-quoting prevents variable interpolation (i.e. it treats strings literally) while double-quoting allows it. Tripping over quoting is a common source of errors, even with intermediate level bash users.

4.2.4 The PATH variable

Another key variable you can see in the list produced by `env` is `PATH`. It's a critical variable for configuration. It tells the shell where to look for programs you try to execute. Try

```
echo $PATH
```

You'll see a colon-separated list of paths. Every program you successfully execute will be found in one of those paths. You can add to this list if you want bash to look somewhere new in addition like this:

```
PATH=$PATH:/path/to/my/new/favorite/software
```

That is also generally how you can recursively modify variables, should you need to.

When executing programs it can be really useful to know *which* copy of a program is being executed, as sometimes there will be multiple versions installed on a given system (this is definitely true of Xanadu). You can use the command `which` for this:

```
which ls
which grep
which cat
```

You'll note that each of these is found in a path found in your `PATH` variable. This will be less trivial and more important to be able to check when it comes to software used for data analysis on Xanadu.

4.2.5 Generalizing code

So far we have been executing commands by explicitly referring to files, programs and their parameters. For example:

```
grep -o -P "(?<=[Ss]pecies )[A-Za-z]+" darwin1859.txt | sort | uniq -c | sort -g | tail
```

In this case, we have specified programs by name, several options, a regular expression, and an input file. We could easily make this line of code more general by replacing important parts of it with variables:

```
REGEX="(?<=[Ss]pecies )[A-Za-z]+"
INFILE=darwin1859.txt

grep -o -P "${REGEX}" $INFILE | sort | uniq -c | sort -g | tail
```

Note that quoting the regular expression variable: `"${REGEX}"` is necessary in this case. If you try without the quotes, some of those special characters will trip bash up, and by extension cause grep to fail.

Using variables like this can be especially useful in two cases:

1. When paths get extremely long and cumbersome and/or file names are not descriptive. Long paths can make code difficult to read. Obscuring them with descriptive variables allows you (and anyone else) to quickly see HOW you ran a program, instead of having to scan over multiple instances of `/the/very/long/and/convoluted/path/to/my/precious/input/data.txt`.
2. When code must be repeated many times on many different input files. Copying, pasting and editing the same line over and over again is extremely inadvisable as even the most conscientious people will be prone to mistakes. If we get used to using variables like this, we gain the flexibility to use loops and other means of parallelization that make our code less error prone, faster, and prefigure the way workflow languages we will explore in the next semester require us to think about and write code.

4.3 Loops

It's often the case that we want to do something repeatedly. It may be that we want to do some analysis the same way for multiple input files, or do something with every element of a list, or line of a file. Sometimes, though less often in data analysis, we may want to repeat a process until some condition is met.

One common way to approach this is with a **loop**. A loop generally has the following very general syntax:

```
loop condition
{ command1
  command2
  command3 }
```

In BASH we typically use **for** and **while** loops. A **for** loop iterates over a list of fixed length executing the same code each time. A **while** loop executes code repeatedly until some condition is no longer met.

4.3.1 for loops.

With **for** loops we can iterate over any sort of list. Let's extract mentions of classic Darwin terms "species", "forms", and "varieties" from the Origin:

```
for WORD in species forms varieties
do
    grep "$WORD" darwin1859.txt >"${WORD}"_mentions.txt
done
```

In this syntax we tell bash we’re going to use the variable `WORD`, and for each iteration of the loop, populate it with an element of the list “species forms varieties”. The elements are separated by whitespace. The code to be executed for each iteration of the list is sandwiched between the keywords `do` and `done`. We redirect the output to three separate files named “species_mentions.txt”, etc.

We can specify the list of elements to be iterated over any number of ways. We can use a range of numbers or letters with a shell expansion:

```
for NUM in {1..10}
do
    echo $NUM
done
```

We can use a glob to iterate over files:

```
for FILE in *mentions.txt
do
    head "$FILE"
done
```

4.3.1.1 A digression about BASH arrays

Sometimes we may wish to construct a list outside of the context of the loop itself. It can be useful to store that list in a special type of variable called a **bash array** (or just “array”, but we will deal with other types of arrays later, so we’ll try to be specific). bash arrays are lists that require you to use a special syntax to create and access their elements.

bash arrays can be created simply with parentheses like this:

```
DARWIN=(species forms varieties)
```

Or using `ls` or `find`, or `*` to grab files:

```
DARWIN=(*mentions.txt)
```

```
DARWIN=($(ls *mentions.txt))
```

In this last case the syntax `$(command)` is a command substitution. It means run the command and insert the output in place (in this case, inside the `()` used to define the array). Also note that the array elements will be parsed based on whitespace, so if file names contain whitespace (you should *avoid* this always) this approach will break.

bash arrays are zero-indexed, meaning the first element is element 0. We can access elements with this syntax:

```
echo ${DARWIN[0]}
```

To write out ALL the elements:

```
echo ${DARWIN[@]}
```

To get the length of the array:

```
echo ${#DARWIN[@]}
```

To use this in a loop we can either provide the whole array like this:

```
DARWIN=($(ls *mentions.txt))

for FILE in ${DARWIN[@]}
do
    echo "FIRST TEN LINES OF $FILE -----"
    head $FILE
done
```

Or we can iterate over the indexes like this:

```
DARWIN=($(ls *mentions.txt))

for NUM in {0..2}
do
    echo "FIRST TEN LINES OF ${DARWIN[$NUM]} -----"
    head ${DARWIN[$NUM]}
done
```

4.3.2 while loops.

`while` loops differ from `for` loops in that they evaluate repeatedly until some condition is no longer satisfied. You can use:

```
COUNTER=1

while [ $COUNTER -le 5 ]
do
    echo "Count: $COUNTER"
    ((COUNTER++))
done
```

Where `[ $COUNTER -le 5 ]` is a conditional construct indicating `COUNTER` must be less than 5. If this evaluates to false, the loop ends. [Conditional constructs](#) are a general way to manage when code is executed, and we will cover them more later. `((COUNTER++))` adds one to `COUNTER` every time the loop executes.

You can also use `while` to iterate over lines in a file:

```
FILE=darwin1859.txt
COUNTER=1

while IFS= read -r line ; do
    echo "Processing line $COUNTER: $line"
    ((COUNTER++))
done < $FILE
```

In this case we're using `< $FILE` to direct the standard input to `FILE` and `while` will read from it. When `FILE` runs out of lines, the loop will break.

4.4 Editing Files

We've covered lots of basic bash features. We're almost ready to start writing scripts. Now that you've been writing lots of commands in the terminal, and you recognize you're connected to a remote computer cluster that your local file system doesn't have access to, you're probably starting to wonder, how do I actually write and save lots of code on the remote server?

In this section we'll deal with that issue and cover some tools you can use to write and edit code, both locally and remotely.

4.4.1 Command-line text editors

The most straightforward approach to this is to use CLI text editors that are already available on Xanadu (and most Linux systems). Since you run them on Xanadu, they can directly create and access files. The most common ones are `nano`, `vim` and `emacs`. `vim` and `emacs` are very powerful editors. They are highly customizable and have large user communities. They have

steep learning curves, however, and there are alternatives to CLI editors, so in this course we are going to focus on **nano**, which is simple to use and suitable for quick edits and copy-paste operations.

If you simply type **nano** on the command line, it will open a new text document. You can immediately begin typing whatever you like. When you're done, you can type **ctrl-x** and it will ask you if you want to save the file. If you do, type **y**, then when prompted, write a file name and hit **enter**. That's it. If you want to edit an existing file, type **nano filename**. Save and exit the same way. **ctrl-c** cancels.

You can navigate around with the arrows on your keyboard (and some keyboard shortcuts we'll talk about in a video). That's it.

4.4.2 Code editors

Another approach suitable for users starting up is to write code using a locally installed, dedicated code editor such as [Sublime Text](#). There are others, but for this course I recommend you download and use Sublime for at least some cases. Unlike **nano**, it has lots of features and user-created plugins. It has **syntax highlighting**, which means if you select the language you are writing in from a dropdown menu in the bottom right corner, it will recognize syntactic features of the language and change the color of the text in ways that greatly help with editing.

A straightforward way to use Sublime to write scripts is to write code locally, and then paste it into documents using **nano**, or use **scp/rsync** to upload the documents to Xanadu. These solutions are not exactly ideal, but they work in a pinch. It is possible to connect Sublime through an ssh tunnel to give it access to documents on Xanadu, but we haven't yet shown you the tools to do that, and we'll introduce another, easier approach in a moment.

4.4.3 Integrated development environments.

IDEs are dedicated code editors with *lots* more features. [Visual Studio Code](#) is another program we recommend you download and install locally. It also has syntax highlighting, but you'll notice you can open up projects and see the entire directory structure, and if you want you can even open a terminal inside it.

4.4.4 Conveniently accessing files.

Ok, so aside from using **nano** (or learning to be a **vim** power user), how can you conveniently access and edit files on Xanadu? There are two relatively straightforward ways:

1. Visual Studio Code has an extension [Remote - SSH](#). This is **highly** recommended. You can connect VS Code to Xanadu directly via SSH. You can open windows focusing on specific directories, visualize the directory structure, and create and edit files directly on Xanadu. You can also open a Xanadu terminal window so that you can test and execute code, all in VS Code.

 Warning

Please don't use language-specific code extensions (python, R, etc) to run code on Xanadu from within VS Code. We'll cover this in the next chapter, but VS Code connects to a **login** node, which has few resources and is shared by many users. To run code on Xanadu, it must be submitted through the batch scheduler, **SLURM**. A VS Code extension that runs code for you can only run it on the login node, which will cause problems for everyone.

2. You can mount the Xanadu file system on your local computer. You can get your operating system to "see" Xanadu's filesystem, and access files as if they were local. You can then edit them using Sublime, VS Code, or anything else. To do this, you **must** be connected to the CAM VPN ([instructions](#)).
- From a mac, you can select from your top dropdown menu **Go:Connect to Server** and enter `smb://cfs09.cam.uchc.edu/home/FCAM/<username>`. When prompted, enter your CAM credentials (not your netID/password).
 - From Windows, you can map a network filesystem using [these directions](#) and the address formatted like this: `\\cfs09.cam.uchc.edu\home\FCAM\<username>`

 Warning

You should only use dedicated software for editing code (or CLI editors on Xanadu). Editing using word processors (such as MS Word) will often lead to difficult to diagnose problems. Word processors often insert hidden characters and/or use line break characters incompatible with Linux. These will cause confusing errors when executing code. The first place to start if you think this might be the issue is to try `cat -A myscript.sh`. If `myscript.sh` contains any weird hidden characters, this will print them. Compare to a script you know works. Non-standard line breaks are often the culprit and this will reveal them.

4.5 Scripts

Ok, now we've got all the pieces, we can start writing longer bits of code into **scripts** so that we can run them as batches instead of interactively, line-by-line. Scripts can be as simple as a series of commands that you *could* execute interactively, or as complicated as computer programs that run entire analyses for you.

In this course, we'll aim for the simpler end of the spectrum. We want to write code in chunks that we can pass off to Xanadu's job scheduler and that will serve as the most fundamental documentation of the analyses we do. While keeping notes on what you do (or try) is important, the code itself is the ultimate documentation of your analysis, so it should be complete and well organized into scripts.

4.5.1 The shebang

A script can be written in any language, so typically the way we start one off in a Linux environment is to tell bash what interpreter to use. Right now we're writing bash code (rather than R, python, perl or something else), so we need to tell bash that. To do that, the first line of the script will always be what's referred to as a **shebang**, or **#!** followed by the interpreter we wish to use, in this case `/bin/bash` so:

```
#!/bin/bash
```

If we were writing python code we might write:

```
#!/bin/env python
```

This would tell the shell to use whichever python interpreter was found by searching our `PATH` variable.

After the shebang line, we can start writing code.

4.5.2 Comment lines

Despite our best efforts, code can sometimes be complicated and confusing to read. Because of this, it's really important to document your code with **comments**. In shell scripting, you can include lines prefixed with `#`, that will be ignored by the interpreter. You can and should write notes on these lines explaining what the code does.

```
#!/bin/bash
```

```
# this script will print the 10 most common words found in a text file and their frequency

# this line specifies the text file
ORIGIN=darwin1859.txt

# this line extracts and prints the word list
grep -o -P "\b[A-Za-z]{4,}\b" $ORIGIN | sort | uniq -c | sort -g | tail -n 10
```

Create this script with the title `commonWords.sh` on Xanadu and change the permission so that you can execute it.

4.5.3 Executing scripts

There are a few ways to execute scripts.

1. `source commonWords.sh` This method *ignores* the shebang (it begins with `#` after all) and executes all the code in the current shell session as if you had typed it on the command line. If you created any variables in the current environment they will be available in the script.
2. `bash commonWords.sh` This method also ignores the shebang and explicitly invokes `bash`, but it creates a *subshell*. The execution context is mostly isolated from the environment you submitted the script, and environment variables you create are not available unless you export them.
3. `./commonWords.sh` This will execute the script, and looks for the shebang to see which interpreter should be used. It also creates an isolated execution context.
4. Passing the script to a job scheduler (in our case SLURM). This will be covered in then next chapter.

What does it mean to have an isolated environment? Modify the `commonWords.sh` script so that the variable definition is *commented out* and thus ignored by the interpreter:

```
#!/bin/bash

# this script will print the 10 most common words found in a text file and their frequency

# this line specifies the text file
# ORIGIN=darwin1859.txt

# this line extracts and prints the word list
grep -o -P "\b[A-Za-z]{4,}\b" $ORIGIN | sort | uniq -c | sort -g | tail -n 10
```

If you try to execute it, you will find that it hangs indefinitely because the `ORIGIN` variable is empty.

If you try to define the `ORIGIN` variable and then run the script it will work with the `source` method:

```
ORIGIN=darwin1859.txt
source commonWords.sh
```

But for it to work with the other two methods, you need to `export` the variable:

```
export ORIGIN=darwin1859.txt
./commonWords.sh
```

Exporting the variable makes it so that it will be inherited by any processes spawned by the current shell.

Another big difference between `source` and the other two methods is that any variables created by the script will be available in the current shell session after the script has completed.

4.5.3.1 Command-line arguments

Note that we can generalize our script by writing it so that it expects command-line input from the user:

```
#!/bin/bash

# this script will print the 10 most common words found in a text file and their frequency

# this line specifies the text file
ORIGIN="$1"
REGEX="$2"

# this line extracts and prints the word list
grep -o -P "$REGEX" $ORIGIN | sort | uniq -c | sort -g | tail -n 10
```

Save this script to the file `test.sh` and make it executable. We can now run it like this:

```
./test.sh darwin1859.txt "\b[A-Za-z]{4,}\b"
```

The variables `$1` and `$2` are automatically parsed as whitespace separated arguments provided on the command line when executing the script. There are more complex ways to provide and parse command-line arguments, but we won't cover them here.

4.5.4 Errors and exit codes

Code can fail for many reasons. You will inevitably make mistakes in command line usage for some program, refer to paths that don't exist, or use incorrect syntax, and an error will result. Computer clusters are complex, and they also inevitably have problems. Sometimes a compute node will crash. Sometimes a user will accidentally get around whatever guardrails the system administrators have in place and muck up some important settings. Troubleshooting these errors can be a major challenge for beginners.

Here are a few common errors you are likely to encounter:

“No such file or directory”

```
cat /path/does/not/exist.txt
```

Leads to `ls: cannot access /path/does/not/exist.txt: No such file or directory.`

Check your path/filename.

“Permission denied”

```
echo "I'm going to put a file in the root directory of the cluster!" >/file.txt
```

Leads to `bash: /file.txt: Permission denied`

Check that your path is correct and that you actually *should* have permission. If you own or share the file or the directory, fix the permissions. If someone else owns it, ask them to fix the permissions.

bad quoting

```
grep endless forms darwin1859.txt
```

Leads to search results for “endless” but also `grep: forms: No such file or directory.` This is a quoting issue. `forms` is treated as another file to be searched because of the whitespace. It should be `grep "endless forms" darwin1859.txt`

“Command not found”

```
gep "endless forms" darwin1859.txt
```

Leads to `bash: gep: command not found.` Classic typo. Command not found can also be a sign of incorrect quoting, or the dreaded incorrect line breaks that arise from editing code in non-code-specific editors.

When a command is evaluated, bash produces an **exit code** and stores it in the special variable `$?`. If the command executes without error, the exit code will be 0. If there is an error it

will be greater than 0. With scripts set up as we have above, the exit code will refer to the *last command* in the script. So if there are two commands in a script, and the second one completes successfully, but the first fails, the exit code will still be 0. By default the script won't just quit when it hits an error. There are [settings](#) that can change the behavior of bash when it encounters errors, but there is [some](#) controversy about whether you should use them or not.

“There is some controversy” is often the case with bash scripting. BASH is a wonky language, and it takes *years* to truly become an expert in it. When learning to do data analysis, we need to do our best, be vigilant about errors, and be open to learning new ways of doing things when we discover problems with our established habits. Pages like [this](#) are great for intermediate bash users to consider, but we can't wait to analyze our data until we've assimilated all these best practices.

4.6 Exercises

5 SLURM: the job scheduler

Learning Objectives:

Query SLURM to determine resource availability
Submit work and a resource request to be managed by SLURM
Monitor the progress of work on SLURM
Evaluate the results of work submitted to SLURM
Start an interactive session on a compute node

5.1 High performance computing review

Now that we've covered connecting to a remote server, basic command-line interface usage, and the fundamentals of writing a script, we're ready to explore how to actually use the Xanadu computer cluster. You should be familiar with this already from ISG5301, but Xanadu is not a single computer. It is a large number of computers networked together, most of which are far more powerful than a standard consumer-grade laptop or desktop. These individual computers are often referred to as **nodes**. Each of the nodes is connected to a **network-attached file system** (NFS) which stores user data. Because all the nodes are connected to this file system, no matter which of the nodes you may be working on at any given moment, you will have access to your data.

There are three main categories of nodes.

1. **Compute nodes:** These are the workhorses of the cluster. They typically have many CPUs (24-96) and memory ranging from 256G to 2TB (even a good consumer laptop won't usually have more than 8 CPUs 16G of memory).
2. **Login nodes:** These are much smaller computers, often with only 2 CPUs and 8G of memory. They may not even be physical machines, but virtual ones. They are meant to serve as portals to more powerful resources. When users connect to Xanadu, they are assigned to one of several login nodes.
3. **The head node:** A head node is typically used only by system administrators to manage the system. It probably runs the workload manager.

Thus far, when we have connected to Xanadu we have connected to a **login** node. Because login nodes have few resources, which are shared among many users, **you should not use them**

for analysis. The kind of light work we have done (navigating the file system, inspecting files) is suitable for the login node, but if you ever run the kind of command that has you thinking, “I’ll just check social media for a moment while this runs” then you have **far exceeded** what you should be doing on a login node.

5.2 SLURM

In order to request suitable resources for data analysis, we need to appeal to software that we will variously refer to as the **workload manager** or **job scheduler**. On Xanadu, we use the software **SLURM**. It is very commonly used on HPCs, but there others such as **PBS** and **LSF**. From a user perspective, these systems have many features in common, and in fact, Xanadu is set up to interpret **PBS** commands if necessary, though this is not recommended.

In this chapter we will cover how to use **SLURM** to ask what resources are available, request resources to do work, monitor the status of running jobs, and evaluate jobs when they have completed.

5.2.1 The general approach

When you have to do some computational work that requires cluster resources (and you will very soon), the process looks something like this:

1. Decide what resources are needed to do the work.
2. Check to see whether the resources are available (whether they exist all, or are currently busy).
3. Submit the work with a resource request to the job scheduler (or simply request resources in the case of interactive work).
4. Monitor job progress until completion or failure (or if interactive work, do the work).
5. Evaluate the work.
6. (Possibly) Go back to (1) for the next step of your analysis.

We’ll cover each of these below.

5.2.2 What resources are needed?

We will cover this question on a job-by-job basis when we start analyzing data as it can be somewhat complicated.

5.2.3 What resources exist/are available?

The two primary SLURM commands used here are `sinfo` and `squeue`. We can also look at the file `/etc/slurm/slurm.conf` to see a list of node *features* that we can request if necessary.

5.2.3.1 `sinfo`

`sinfo` prints information about available compute nodes and their status. At the moment of writing, running `sinfo` with no options prints the following output:

PARTITION	AVAIL	TIMELIMIT	NODES	STATE	NODELIST
general*	up	infinite	1	drain	xanadu-05
general*	up	infinite	23	mix	xanadu-[01,03-04,08,10,25,39,50-52,57-61,64-66,69-72]
general*	up	infinite	4	alloc	xanadu-[02,62-63,67]
general*	up	infinite	5	idle	xanadu-[46-47,49,53-54]
vcell	up	infinite	4	drain	xanadu-[78-81]
vcell	up	infinite	3	idle	xanadu-[76-77,82]
vcellpu	up	infinite	1	idle	xanadu-32
himem	up	infinite	2	mix	xanadu-[40,44]
himem	up	infinite	2	idle	xanadu-[06,43]
himem2	up	infinite	2	mix	xanadu-[07,75]
xeon	up	infinite	1	drain	xanadu-05
xeon	up	infinite	18	mix	xanadu-[03-04,08,39,50-52,57-61,64-66,69-70,72]
xeon	up	infinite	4	alloc	xanadu-[02,62-63,67]
xeon	up	infinite	5	idle	xanadu-[46-47,49,53-54]
amd	up	infinite	2	mix	xanadu-[10,25]
mcbstudent	up	infinite	2	mix	xanadu-[68,71]
gpu	up	infinite	1	drain	xanadu-05
gpu	up	infinite	5	mix	xanadu-[01,03-04,07-08]
gpu	up	infinite	1	alloc	xanadu-02
gpu	up	infinite	3	idle	xanadu-[06,84-85]
crbm	up	infinite	2	idle	xanadu-[55-56]

Nodes on clusters are sometimes, though not always divided into **partitions**. If they are, you must decide which partition you want to submit to. Partitions need not be mutually exclusive, and may have different behaviors and limitations. Key partitions on Xanadu are **general** and **himem**. In the basic `sinfo` output, you see nodes in each partition, divided into various categories, and a listing of the nodes in each category: **alloc** = allocated, i.e. all resources on these nodes have been assigned; **mix** = some resources on these nodes have been assigned and some remain available; **idle** = these nodes are currently unused; **drain** = these nodes are not accepting new jobs (probably they will be rebooted when current jobs finish). Xanadu

actually has many more nodes than this, at the time of writing many had been removed for maintenance.

You will frequently want to see more detail than this. The following command will print information for each node, formatted according to the obscure syntax found in the `sinfo` man page.

```
sinfo --format="%10P %6t %15D %15C %15F %10m %10e %15n %30E %10u"
```

The first few lines of output:

PARTITION	STATE	CPU_LOAD	CPUS(A/I/O/T)	NODES(A/I/O/T)	MEMORY	FREE_MEM	HOSTNAME
general*	drain	0.01	0/0/36/36	0/0/1/1	225612	16760	xanadu01
general*	mix	18.37	28/8/0/36	1/0/0/1	257669	9569	xanadu02
general*	mix	11.49	35/1/0/36	1/0/0/1	257845	45859	xanadu03
general*	mix	8.34	34/2/0/36	1/0/0/1	257845	1307	xanadu04
general*	mix	2.41	9/27/0/36	1/0/0/1	257669	50949	xanadu05
general*	mix	10.76	15/33/0/48	1/0/0/1	386972	52520	xanadu06
general*	mix	12.61	44/4/0/48	1/0/0/1	257949	7961	xanadu07
general*	mix	6.03	8/8/0/16	1/0/0/1	128825	26234	xanadu08
general*	mix	0.06	8/32/0/40	1/0/0/1	257914	119938	xanadu09
general*	mix	0.08	8/32/0/40	1/0/0/1	192032	13437	xanadu10

Here you get lots of useful information. The column `CPUS(A/I/O/T)` tells you how many cpus are **allocated/idle/other/total** on each node. You can also see the amount of memory and free memory (in megabytes) on each node. This can give you a pretty fine-grained sense of what resources currently exist in terms of CPU and memory, and how many resources are **idle** on the cluster.

5.2.3.2 `squeue`

The `squeue` command lets users see what jobs have been submitted to the job queue, what their status is, and why. This includes pending jobs that are waiting for resources to become available, and currently running jobs. Depending on SLURM configuration, this command may show *only* jobs you have submitted, or it may show *every* job. At the time of writing, Xanadu was configured to show *every* job.

Some example output from `squeue`:

JOBID	PARTITION	NAME	USER	ST	TIME	NODES	ODELIST(REASON)
7949897	general	nf-MAIN_	ebrannan	PD	0:00	1	(QOSMaxMemoryPerUser)
7949952	general	nf-MAIN_	vvuruput	PD	0:00	1	(Resources)

7949954	general	nf-MAIN_	vvuruput	PD	0:00	1 (Priority)
7953478	general	slow59.s	mhossein	PD	0:00	1 (Priority)
7953479	general	slow60.s	mhossein	PD	0:00	1 (Priority)
7946600	general	busco	shillima	R	2-20:56:08	1 xanadu-52
7945409	general	bash	shird	R	3-01:21:06	1 xanadu-74
7917445	himem	R_divers	pmartine	R	6-01:31:02	1 xanadu-40
7953261	mcbstuden	bash	meds5420	R	3:46:07	1 xanadu-68

The JOBID is a unique numerical identifier assigned by SLURM to *every* job it runs. This is important information we'll discuss later. NAME is the name the user gave SLURM for the job (truncated in this view). USER is user who submitted the job. ST is the status: PD = pending; R = running. NODLIST(REASON) is either the list of compute nodes the job was assigned to (e.g. xanadu-52 for job 7946600) or the reason the job is not yet running: "Priority" essentially means the job will start at any moment; "Resources" means the requested resources are busy; "QOSMaxMemoryPerUser" means the running the job would exceed the user's maximum memory allotment. Individual users have resource limits so that no one person can dominate the system.

You can use the flag `-u <user>` to restrict to a given userid. You can try `squeue -o "%.12i %.9P %.30j %.8u %.2t %.10M %.6D %R"` for a little more spacious formatting.

5.2.3.3 /etc/slurm/slurm.conf

SLURM has a configuration file `/etc/slurm/slurm.conf` that lists features of each node. Some of this is listed by `sinfo`, but occasionally you may run into software that has very specific needs. Something may require a particular *instruction set* be present on the CPUs, or you may require a particular type of GPU be present on the node. These **features** are listed at the end of this file, and these features can be requested using **feature constraints**. Try `cat /etc/slurm/slurm.conf`. Look near the end of the file at the section beginning `# COMPUTE NODES`:

COMPUTE NODES

```

NodeName=xanadu-01 CPUs=36 SocketsPerBoard=2 CoresPerSocket=18 ThreadsPerCore=1 RealMemory=256G
NodeName=xanadu-02 CPUs=36 SocketsPerBoard=2 CoresPerSocket=18 ThreadsPerCore=1 RealMemory=256G
NodeName=xanadu-03 CPUs=36 SocketsPerBoard=2 CoresPerSocket=18 ThreadsPerCore=1 RealMemory=256G
NodeName=xanadu-04 CPUs=36 SocketsPerBoard=2 CoresPerSocket=18 ThreadsPerCore=1 RealMemory=256G

```

Here you can see that node xanadu-01 has 36 CPUs, ~256G of memory, a Xeon cpu and an NVIDIA A10 GPU.

5.2.4 Request resources (and submit work)

So far we have shown you ways to ask questions about the cluster and its resources. Now we'll cover how to request them to get actual work done. There are two common use cases for analyzing data using cluster resources: running batch scripts that require no active user intervention, and doing interactive analysis.

5.2.4.1 Running batch jobs with `sbatch`

When you run a batch job, you write a script that will run all the steps of a given analysis for you, hand that script to SLURM, and wait for it to complete (or fail). Using one command (or one script) you tell SLURM which resources you need, and what you want it to do. The command we use for this is `sbatch`.

Running `sbatch` is in essence, just like running a script as we did in the previous chapter, except that instead of the script being run by the current shell, you hand it to SLURM with your resource request, SLURM puts it in the job queue, and when a node (or nodes) with enough resources becomes available, assigns the job there and runs it for you.

Let's look at the most basic usage using our fully defined script from the previous chapter:

```
#!/bin/bash

# this script will print the 10 most common words found in a text file and their frequency

# this line specifies the text file
ORIGIN=darwin1859.txt

# this line extracts and prints the word list
grep -o -P "\b[A-Za-z]{4,}\b" $ORIGIN | sort | uniq -c | sort -g | tail -n 10
```

Save it to `commonWords.sh`. Note that because we are passing this file to SLURM, not directly to bash, we *definitely* need the shebang at the top.

On Xanadu, you must minimally specify the partition you want SLURM to run the job on, and its associated quality of service (or QOS). Not all SLURM clusters require a QOS to be specified, but Xanadu does.

To submit the script to be run on the general partition you can do:

```
sbatch -p general --qos=general commonWords.sh
```

Our script writes to stdout. Where did the results go? By default, to a file named `slurm-JOBID.out`. In my case at the time of writing `slurm-7953521.out`. You can inspect this file and the results should be there.

This basic approach will request a minimal amount of memory and number of CPUs which will be too little for most real data analysis. We can request more with some options:

```
SBATCH -p general --qos=general -c 12 --mem=10G commonWords.sh
```

Now we're asking for 12 CPUs with `-c` and 10 gigabytes of memory with `--mem=10G`. Far more than is needed for this tiny job. What happens if you request more memory than is available on general partition?

```
SBATCH -p general --qos=general -c 12 --mem=1000G commonWords.sh
```

An error!

```
SBATCH: error: Memory specification can not be satisfied
SBATCH: error: Batch job submission failed: Requested node configuration is not available
```

This is relatively straightforward. But where does stderr go? Can we direct that to a file? Can we name the jobs so we can see them in the queue? Or rename these `slurm-JOBID` files so that they are a little easier to sort through when we run are running lots of jobs?

The answer is yes, but you can imagine our command-line is going to start getting very long and cumbersome. To solve this, we typically specify SLURM options in a **header** at the top of our script.

Let's put one in our script:

```
#!/bin/bash
#SBATCH --job-name=commonWords
#SBATCH -n 1
#SBATCH -N 1
#SBATCH -c 10
#SBATCH --mem=20G
#SBATCH --partition=general
#SBATCH --qos=general
#SBATCH --mail-user=MY_EMAIL@uconn.edu
#SBATCH --mail-type=ALL
#SBATCH -o %x_%j.out
#SBATCH -e %x_%j.err
```

```
hostname
```

```
date
```

```
# this script will print the 10 most common words found in a text file and their frequency

# this line specifies the text file
ORIGIN=darwin1859.txt

# this line extracts and prints the word list
grep -o -P "\b[A-Za-z]{4,}\b" $ORIGIN | sort | uniq -c | sort -g | tail -n 10
```

Any command-line option we can pass to `sbatch` can also be placed in a header that immediately follows the shebang. Each header line begins with `#SBATCH` and a command-line flag follows. If we need to, we can override any of these header lines with new values on the command-line, without altering the script.

Here we name the job `commonWords`, request 10 cpus and 20G of memory. The mail options are optional, but if you provide an e-mail, then SLURM will let you know when your job starts and ends. The last two options specify the file name format for any output written to stdout or stderr. The format is `jobname_jobid{.out,.err}`

We also specify a few other options that aren't necessary and that we rarely change: `-n 1`, saying that we want SLURM to launch 1 task, and `-N 1` saying that we want the resources for the task to be on a single node. These will suit all of our work in this course.

You may also notice we added two extra lines to the top of the script `hostname` and `date`. These will write out which compute node the job ran on, and the date it began to the stdout. This can be helpful information if you need to seek help when troubleshooting. Sometimes individual nodes on the cluster, rather than user errors, can be the source of problems.

Update `commonWords.sh` with this header and run it again. Now you can simply enter:

```
sbatch commonWords.sh
```

Now, instead of seeing a single file `slurm-<JOBID>.out`, you should see a pair of files: `commonWords_<JOBID>.out` and `commonWords_<JOBID>.err`. The `.out` file should contain the output from our script, along with the results of `hostname` and `date`, and `.err` should be empty (unless there were any errors).

So, to sum up we request resources and submit work to be run on the cluster by SLURM. Putting SLURM options in the script header is extremely useful, as it keeps a record of *how* we ran our code, not just what code we ran.

5.2.4.2 Starting interactive sessions with `srun`.

It is sometimes the case that we want to do analysis on the cluster that is more intensive than what is permissible on the login nodes, but we are unable to write all the steps into a batch script. This might be an exploratory analysis, where we don't know what the steps are yet, or we might be putting together a complicated set of piped commands and we want to test that the pipe is doing what we expect before we run the entire job.

For this we use an **interactive session**. In an interactive session, we request resources on a compute node, and SLURM drops us into a bash session on that node, rather than a login node. On the compute node we no longer have to worry about gumming things up for everyone else. SLURM will not let us exceed our CPU request, and if we run a program that attempts to exceed the memory request, the session will simply be canceled.

We use the `srun` command for this. The syntax for requesting an interactive session is:

```
srun -p general --qos=general -c 2 --mem=10G --pty bash
```

Try it. To ensure that you have successfully started an interactive session, type `hostname` at the prompt. You should see the name of a compute node that can be found in `sinfo`, something like `xanadu-01`, rather than a login host name like `hpc-ext-1`.

You can exit back to the login node with `exit`

5.2.5 Monitor job progress

There are several strategies for monitoring job progress. We will talk about them here, and practice them later when we start doing longer-running jobs.

1. Check `squeue` to see if your job is pending, currently running, or has completed (or failed) and no longer in the queue.
2. Monitor output files. Check the stdout and stderr log files captured by SLURM. Many programs write progress messages or errors to these files. Check the output files themselves as well.
3. Log in to the compute node and run `top` to see how/if your process is running.

5.2.5.1 `squeue` again

When the cluster is busy and jobs are large enough that they wait in the queue for a while before resources are available, you can use `squeue` as we did above to see what their status is. If they are running, you'll also be able to see how long they've been running. Jobs that complete too quickly or too slowly are sometimes a sign that something isn't right (or that your expectations aren't right).

5.2.5.2 Monitor output files

You can monitor the `.out` and `.err` files, and output or log files produced by your script or the program you're running. In all of these cases, the first thing you can do is simply use `ls -l` and look at the time stamps. Are any of these files being updated?

Depending on the program or the nature of your script, the `.err`, or `.out` files are going to be key files to check. These are updated in real time. If there are errors, warnings, or progress messages, they are likely to be in one of these two files. Some programs will write helpful, succinct messages, and some will write an endless alphabet soup that makes it hard to distinguish normal progress from problems. It will all depend on the program.

When it comes to program- or script-specific files, you'll have to understand a little about the output you're expecting. Some programs write no output until they're complete, others write out results nearly as fast as they read data in. In the latter case, be sure to check on these files at least once and ensure that they're growing as the program is running.

5.2.5.3 Log in to your compute node

There are lots of cases where you might want to get a more direct look at how your job is going. SLURM doesn't have good tools for this. The solution here is to actually log directly in to the compute node and have a look. First, check which node your job is running on using `squeue`. Then you have two options to get into the node.

First, you can use `srun` and request the node with, e.g. `-w xanadu-01`:

```
srun -w xanadu-01 -p general --qos=general -c 1 --mem=500G --pty bash
```

Here we request minimal resources so that we are more likely to quickly get the session started.

Second, if the node is fully subscribed and you can't get an interactive session, you can still get in from the login node with:

```
ssh xanadu-01
```

If you go this route, you should not do **anything** else other than check on your job. If you couldn't get an interactive session on your node, then all CPU or all memory has been requested, and you are essentially oversubscribing the node by logging in this way. You should check on your job quickly and type `exit` to log out.

With either method, once you're on the compute node, you can use the program `top` to see running processes. You will see ALL running processes on the node in a constantly updating list, including user and system processes. To see your own processes type `u` and then your username followed by `enter`. To sort processes by memory usage, type `shift-m` and by

processor usage `shift-p`. `%CPU` refers to the percentage of a single CPU, so if you requested 10 cpus, you may see up to `%1000` CPU usage. `%MEM` refers to the percentage of the total memory on the node. The `S` column indicates whether your process is running (R) or sleeping (S). In an ideal world, whatever you're running would be using all the CPU and all the memory you requested. In reality this is rarely the case. Resource usage often fluctuates. If you see that your processes are often sleeping, that can be a sign they aren't running efficiently and that something may be wrong. If you see that processes you are running never seem to be using as much CPU or memory as you requested, it may be that you requested too much, or that you forgot to tell the actual program you're running how many CPUs to use, for example.

Type `q` to quit `top`.

5.2.6 Evaluate the work

When the job is no longer in the queue, it has completed, failed, or been canceled. You first need to figure out which this is! The two main approaches are to use the slurm commands `seff` and/or `sacct`, and to look at the same output files you looked at when monitoring the job's progress.

5.2.6.1 `seff` and `sacct`

`seff` is the simplest approach. Simply type `seff <JOBID>` at the command line. You will get a report like this from our `commonWords.sh` job:

```
Job ID: 7953646
Cluster: xanadu
User/Group: nreid/cbc
State: COMPLETED (exit code 0)
Nodes: 1
Cores per node: 10
CPU Utilized: 00:00:01
CPU Efficiency: 10.00% of 00:00:10 core-walltime
Job Wall-clock time: 00:00:01
Memory Utilized: 1.56 MB
Memory Efficiency: 0.01% of 20.00 GB
```

Some of this is self-explanatory at this point. `CPU Utilized` is the sum of the CPU-hours used by the job. `Wall-clock time` means the actual time span over which the job ran. `CPU Efficiency` is `CPU Utilized / (cores per node * wall-clock time)`, so the average rate of CPU usage. `Memory Efficiency` is the **peak** memory usage divided by the memory requested. This job was very simple and used virtually none of the resources we requested. This is *not*

what you generally want to see, but because our job only took 1 second, it's not that big of a deal. If every job on the cluster had efficiencies ranging from 1-10%, it would be a pretty big waste.

`sacct` is pretty large and complicated SLURM function that can extract all kinds of job information. We won't get into it too much here except to say this command can provide some detail on a job:

```
sacct -o jobid%-11,jobname%30,nodelist%15,user%12,group%15,partition,state,ReqMem,MaxRSS,R
```

For this job we get this output:

JobID	JobName	NodeList	User	Group	Part
7953646	commonWords	xanadu-25	nreid	cbc	g

5.2.6.2 Checking output files

The approach is basically the same as above. Check your log files for errors, warnings and progress messages. Check the program outputs to see that they are what you expect. That last bit requires some understanding of what you're trying to do.

5.2.7 What resources *should* I request?

This is a perennially challenging problem. In much of bioinformatics, software developers do not, or perhaps cannot, give general advice about what resources their program will need. There are so many axes of variation for different datasets: experimental design, species, tissue, sample size and more. For a given algorithm, these may drastically impact resource needs, or not impact them at all.

When analyzing a new dataset, doing a new type of analysis, or using a new piece of software, we generally advise people to expect to have to experiment a bit. We will talk about resource requests for different pieces of software as we move through the course.

That said, we have some general guidelines:

1. Check the software documentation. It's quite possible there is a critical variable that defines how much memory or CPU the program requires, or can utilize, and that the developer has actually explained it for you quite clearly.
2. When you run the job for the first time, check the CPU and memory efficiency. See if the job failed because it ran out of memory usually `oom-kill` is somewhere in the `.err` file. Request more memory if so. Request fewer resources if efficiency was low.

3. Remember that when requesting CPUs, you **almost always** have to tell the actual program you are running how many CPUs are available to it. If you tell SLURM you want 10 CPUs, you usually have to provide an option (sometimes `-p` or `-t` or `-c`) telling the program how many it can use.
4. Try to tune your resource requests to your actual analyses. If you just copy-paste the same SLURM header requesting 24 CPUs and 100G of memory for every job, sometimes it won't be enough, and most of the time it will be way too much. Either way wastes resources for everyone, and your time. Bigger jobs often sit longer in the queue. Rerunning failed jobs is a huge pain.

The UConn Computational Biology Core has a document you can reference about resource requests (much of which is covered here) [here](#).

5.3 Exercises

6 Data Storage and Software on HPC

Learning Objectives:

We have two more important topics to cover before we move on to actually starting to analyze some real data.

6.1 Data storage and access

home

scratch

tmp

/isg/shared

6.2 Software

compiling

the module system

conda

containers

Part V

Sequencing Data

7 Untitled

8 seq data

Part VI

RNA-seq

9 Untitled

10